

ZaraiLink Trade Intelligence

Kaavish Report
presented to the academic faculty
by

Syed Fahad Nadeem	sn07558
Umar Kashif	ut07527
Salman Adnan	sa07885
Ibrahim Rana	ir08043



In partial fulfillment of the requirements for
Bachelor of Science
Computer Science

Dhanani School of Science and Engineering

Habib University
Spring 2026

ZaraiLink Trade Intelligence

This Kaavish project was supervised by:

Dr. Faisal Alvi
Faculty of Computer Science
Habib University

Approved by the Faculty of Computer Science on _____.

Dedication

We dedicate this project to our beloved parents, whose love, prayers, sacrifices, and unwavering support have been the foundation of our journey. Their encouragement gave us the strength to keep moving forward through every challenge, long night, and moment of uncertainty. Whatever we have achieved in this work is deeply rooted in the values, resilience, and belief that they have always instilled in us.

We also dedicate this project to our siblings, close friends, and loved ones who stood by us with patience, motivation, and kindness throughout this journey. Their confidence in us, their words of encouragement, and their presence during difficult times helped us remain focused and hopeful when the work became demanding.

A special dedication goes to our team. This project would not have been possible without the trust, collaboration, and shared commitment that brought us together. Each member contributed not only effort and ideas, but also support, understanding, and resilience. The experience of building this project together has been one of the most meaningful parts of the journey, and it is something we will always carry with pride.

Finally, we dedicate this work to Almighty God. We are deeply grateful for the strength, health, patience, and guidance that made it possible for us to complete this project. In moments of pressure and uncertainty, faith gave us the ability to persevere and continue with purpose.

Acknowledgements

We would like to express our sincere gratitude to our supervisor, Dr. Faisal Alvi, for his guidance, encouragement, and continuous support throughout this project. His feedback, insights, and academic direction played a central role in shaping our work and helping us refine both the technical and practical aspects of the system. His mentorship helped us remain focused, think critically, and approach challenges with clarity and discipline.

We are also deeply thankful to Pakistan Agricultural Research and the wider team we worked with for their collaboration and trust. In particular, we would like to acknowledge Lahish and Mubashir for their support, domain input, and engagement throughout the project. Their perspective on real-world trade and agricultural needs helped us align the system with practical use cases and gave valuable direction to the development of ZaraiLink.

We would further like to thank our families, friends, and well-wishers for their constant encouragement and belief in us. Their support came in many forms, from motivation during stressful periods to patience and understanding during long phases of work and revision. Their presence made this journey lighter and more meaningful.

Finally, we would like to acknowledge every person who contributed, directly or indirectly, to the completion of this project. Whether through guidance, technical discussion, feedback, or simple encouragement, each contribution helped us bring this work to completion.

Abstract

ZaraiLink is an AI-assisted trade intelligence platform developed to transform raw import and export trade records into actionable business insights for users involved in commodity and agricultural trade. The motivation for the project arises from the difficulty faced by businesses, especially in developing markets such as Pakistan, in accessing reliable trade intelligence for supplier discovery, buyer discovery, pricing analysis, and market evaluation. Existing trade data is often unstructured, difficult to interpret, and inaccessible to users without technical expertise.

The proposed system integrates structured trade data engineering, hierarchical HS code organization, natural language query interpretation, search aggregation, ranking, and token-based premium access control into a unified platform. Users can search the system through HS codes, product names, or natural language trade queries. Depending on the input type, the platform routes the user either to ranked supplier or buyer results or to a market intelligence dashboard for category-level analysis. The system also includes a verified trade directory that supports company discovery and token-based unlocking of key contact information.

From a technical perspective, ZaraiLink is implemented as a full stack web application using a React frontend, a Django backend, and PostgreSQL for structured data storage. The search engine follows a dual-track architecture in which exact structured queries are handled through a deterministic fast path, while conversational queries are interpreted through a Natural Language Understanding pipeline. This pipeline combines intent detection, keyword extraction, country normalization, and constraint parsing to convert unstructured user input into structured trade filters. Aggregation logic then transforms raw shipment records into company-level intelligence using metrics such as trade volume, shipment count, average price, and activity recency.

The system was validated through automated backend testing, script-based evaluation, and workflow-level testing. Results showed that ZaraiLink successfully supports natural language search, HS code-based intelligence views, access-controlled premium data visibility, and company-level trade discovery. The project demon-

strates that AI-assisted trade intelligence can be made more accessible, practical, and locally relevant for businesses seeking data-driven decision support in agricultural and commodity trade.

Contents

1	Introduction	11
1.0.1	Problem Statement	11
1.0.2	Proposed Solution	12
1.0.3	Intended User	12
1.1	Project Gantt Chart and Deliverables	14
1.1.1	Deliverables for Kaavish I	14
1.1.2	Deliverables for Kaavish II	15
1.2	Key Challenges	16
2	Literature Review	18
2.0.1	Introduction	18
2.0.2	Modern Information Retrieval Systems	19
2.0.3	Query Understanding and Intent Detection	20
2.0.4	Entity Extraction and Structured Query Parsing	21
2.0.5	Query Representation and Keyword Extraction	22
2.1	Existing Trade Intelligence Platforms	22
2.1.1	Tridge	22
2.1.2	Volza	23
2.1.3	TradeForesight	24
2.1.4	Gap Analysis and Motivation	25
3	Software Requirement Specification (SRS)	26
3.1	Functional Requirements	26
3.1.1	User Authentication	26
3.1.2	User Profile & Account Management	27
3.1.3	Dashboard & Landing	28
3.1.4	Trade Search Engine	28
3.1.5	Deal Detail — Supplier & Buyer Intelligence	31

3.1.6	Supplier & Buyer Comparison	33
3.1.7	Trade Directory	33
3.1.8	Company Profile	34
3.1.9	Watchlist	35
3.1.10	Token Wallet & Subscription Plans	35
3.2	Non-Functional Requirements	36
3.2.1	Performance	36
3.2.2	Scalability	37
3.2.3	Security	37
3.2.4	Usability	38
3.2.5	Reliability	38
3.3	Use Cases	39
3.3.1	Registered Member	39
3.3.2	Platform Administrator	40
3.4	User Interfaces	41
3.4.1	Dashboard	41
3.4.2	HS Code Results Dashboard	42
3.4.3	Natural Language Query Results	43
3.4.4	Deal Detail Interface	44
3.4.5	Find Suppliers Directory	45
3.4.6	Find Buyers Directory	46
3.4.7	System Diagram	47
4	Software Design Specification (SDS)	50
4.1	Software Design	50
4.2	Data Design	58
4.2.1	Overview	58
4.2.2	Entity Relationship Diagram	58
4.2.3	Core Data Groups	59
4.2.4	Design Rationale	60
5	Methodology and System Implementation	61
5.1	Methodology	61
5.1.1	System Development Approach	61
5.1.2	Data Preparation and Organisation	62
5.1.3	Trade Search and Query Processing Pipeline	64
5.1.4	Access Control and Token Enforcement	67
5.1.5	Trade Directory and Contact Unlocking	69

5.1.6	Frontend and User Interaction Flow	70
6	Testing and Results	71
6.1	Testing and Results	71
6.1.1	Testing Approach	71
6.1.2	Automated Testing Results	72
6.1.3	Data Validation and Aggregation Results	73
6.1.4	Natural Language Query Evaluation	75
6.1.5	Access Control and Monetization Results	79
6.1.6	Trade Directory and Workflow Validation	81
6.1.7	Observed Limitations During Testing	82
6.1.8	Summary of Results	83
7	Conclusion and Future Work	84
8	Reflection	86
8.1	Reflection	86
8.1.1	Individual Reflection	86
8.1.2	Team Reflection	88
8.1.3	Process Reflection	89
8.1.4	Plans vs Achievements	90
	References	93

List of Figures

1.1	ZaraiLink System Gantt Chart	14
3.1	Use Case Diagram for Registered Member	40
3.2	Use Case Diagram for Platform Administrator	41
3.3	Dashboard interface of ZaraiLink	42
3.4	HS code results dashboard with four trade perspective tabs	43
3.5	Search results for a natural language trade query	44
3.6	Deal detail page for an individual trading entity	45
3.7	Find Suppliers directory interface	46
3.8	Find Buyers directory interface	47
3.9	ZaraiLink System Architecture Diagram	49
4.1	UML Class Diagram for ZaraiLink	51
4.2	Entity Relationship Diagram of ZaraiLink	59
5.1	Trade data preparation and loading pipeline used in ZaraiLink	64
5.2	Multi query processing pipeline for HS code, product, and natural language search	67
5.3	Token based access control workflow for premium data unlocking	69
6.1	Summary view of automated backend testing results	73
6.2	Aggregation result showing filtered and ranked supplier intelligence	74
6.3	Observed accuracy and latency characteristics of the Natural Language Understanding pipeline	78

List of Tables

6.1	Summary of automated testing results	73
6.2	Observed data validation and aggregation results	75
6.3	Natural language query evaluation summary	79
6.4	Access control and monetization validation	81

1. Introduction

1.0.1 Problem Statement

Global agricultural and commodity trade remains highly fragmented, opaque, and inefficient, particularly for small and medium-sized enterprises in developing markets such as Pakistan. Although large volumes of import and export shipment data exist, this data is often raw, inconsistent, and difficult to interpret without technical expertise. As a result, businesses struggle to identify reliable suppliers, compare prices, evaluate market activity, and discover new trade opportunities through data-driven means.

In Pakistan, many exporters, importers, and trading firms continue to rely on informal networks, intermediaries, or static directories when making sourcing and selling decisions. This creates several inefficiencies, including price asymmetry, limited market reach, weak transparency, and increased transaction risk. Businesses often lack access to tools that can transform historical trade records into actionable intelligence such as supplier performance insights, market-level price trends, and evidence-based partner evaluation.

Existing international trade intelligence platforms provide some access to shipment data, but they are often expensive, not tailored to local market needs, and limited in accessibility for smaller firms. They also do not always support flexible search and discovery methods suitable for users with varying levels of technical expertise. There is therefore a clear need for a more accessible and intelligent system that allows businesses to explore trade data, discover new suppliers and buyers, and make more informed commercial decisions.

ZaraiLink is motivated by this gap. It aims to transform raw trade records into a structured, searchable, and insight-driven platform that enables users to access relevant market intelligence in a practical and usable form.

1.0.2 Proposed Solution

ZaraiLink is an AI-assisted trade intelligence platform designed to help users discover, evaluate, and connect with suppliers and buyers through structured trade data and intelligent search. The system combines data engineering, hierarchical product classification, natural language query interpretation, aggregation-based analytics, and token-based premium access control within a unified web platform.

The core of ZaraiLink is a search and intelligence workflow that allows users to query the system using HS codes, product names, or natural language trade requests. Structured queries such as exact HS code searches are handled through a deterministic fast path, while conversational trade requests are interpreted through a Natural Language Understanding pipeline. This enables the system to support both precise and flexible discovery behavior within the same search interface.

The search engine routes users toward one of two principal intelligence outputs. Queries based on natural language or product names are converted into ranked supplier or buyer results. Queries based on HS codes are routed to a market intelligence dashboard that presents category-level insights. These outputs are supported by aggregation logic that transforms raw transaction records into meaningful metrics such as trade volume, shipment count, average price, and activity recency.

In addition to market intelligence, the platform includes a verified trade directory that allows users to explore company profiles, browse suppliers and buyers, and unlock premium contact information. This directory extends the value of the trade dataset by connecting aggregated trade evidence with structured company-level information. A token-based monetization layer governs premium access to trade intelligence and protected contacts, enabling the platform to function within a commercially realistic access model.

The overall system is designed to be practical for real users rather than only technically sophisticated. By combining structured dashboards, ranked search results, company profiles, and controlled premium access, ZaraiLink enables users with limited technical expertise to benefit from data-driven trade discovery and decision support.

1.0.3 Intended User

ZaraiLink is intended for a range of users involved in agricultural and commodity trade who require better access to market intelligence and partner discovery tools. The system is especially relevant for users operating in data-constrained or network-dependent trade environments.

The primary user groups include:

- **Small and Medium-Sized Enterprises (SMEs):** SMEs can use ZaraiLink to identify new suppliers and buyers, compare pricing patterns, and reduce dependence on informal intermediaries.
- **Exporters and Importers:** These users can analyze shipment histories, evaluate trade partners, and improve sourcing or selling strategies using structured trade evidence.
- **Trading Companies and Brokers:** Intermediaries can use the platform to improve market visibility, identify commercial opportunities, and support clients with better-informed trade recommendations.
- **Market Analysts and Researchers:** Analysts can use the platform’s aggregation and filtering capabilities to study trade flows, pricing behavior, and commodity-specific market patterns.
- **Government and Policy Stakeholders:** Public-sector and regulatory stakeholders can use trade intelligence outputs to understand import and export trends and support evidence-based policy planning.

Each of these user groups interacts with the platform differently. Some may focus on supplier discovery and contact access, while others may be more interested in product-level dashboards, company reliability, or trade trend analysis. ZaraiLink is designed to support these varied use cases through a modular but unified system interface.

1.1 Project Gantt Chart and Deliverables

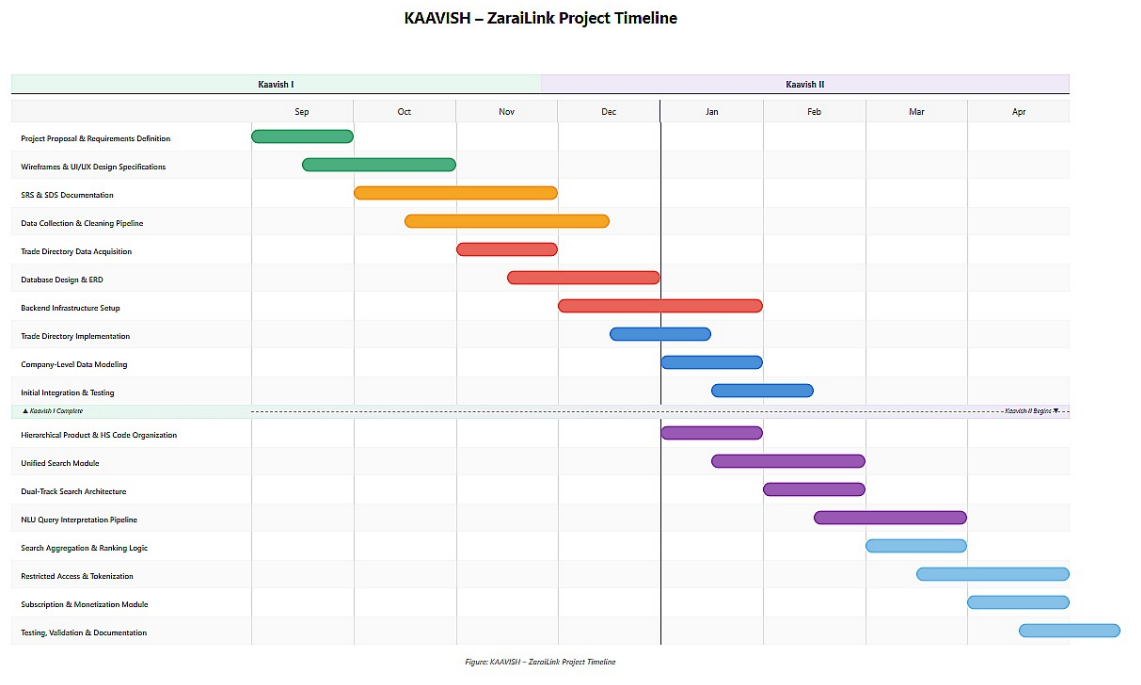


Figure 1.1: ZaraiLink System Gantt Chart

1.1.1 Deliverables for Kaavish I

The following deliverables were completed during the initial phase of the project:

- Project Proposal and Requirements Definition:** A detailed document outlining the project objectives, scope, system vision, and initial functional requirements.
- Wireframes and UI/UX Design Specifications:** Low-fidelity and high-level design layouts for key system interfaces, including the trade discovery flow and dashboard structures.
- System Requirements Specification (SRS) and System Design Specification (SDS):** Comprehensive documentation covering system functionality, architecture, and design decisions.

4. **Data Collection and Cleaning Pipeline:** Collection, cleaning, and standardization of one year of import and export trade data to ensure consistency, accuracy, and usability for downstream processing.
5. **Trade Directory Data Acquisition and Structuring:** Aggregation and validation of company directory data from multiple external sources, ensuring a consistent and reliable dataset for company profiling.
6. **Database Design and Entity Relationship Diagram (ERD):** Design and implementation of a normalized database schema representing trade data, company profiles, and relationships between entities.
7. **Backend Infrastructure Setup:** Initialization of the backend system using Django and PostgreSQL, including core models, APIs, and data pipelines.
8. **Trade Directory Implementation:** Development of the company directory module, enabling users to browse and explore structured company profiles.
9. **Company-Level Data Modeling:** Processing and aggregation of transaction data to derive company-level insights such as trading partners, traded products, and activity patterns.
10. **Initial Integration and Testing:** Basic integration of frontend, backend, and database components, along with preliminary testing to validate system functionality.

1.1.2 Deliverables for Kaavish II

The following deliverables were completed during the second phase of the project:

1. **Hierarchical Product and HS Code Organization:** A multi-level product hierarchy was implemented to organize trade data by HS chapter, category, subcategory, and specific product item, enabling drill-down analytics and category-based access control.
2. **Unified Search Module:** A single search interface was developed to support HS code queries, product-name searches, and natural language trade queries through one integrated workflow.

3. **Dual-Track Search Architecture:** The search engine was designed with a deterministic fast path for exact HS code and category-based lookups, together with a Natural Language Understanding pipeline for conversational and ambiguous user queries.
4. **Natural Language Query Interpretation Pipeline:** A multi-stage NLU pipeline was implemented using intent classification, entity extraction, keyword extraction, fuzzy country normalization, and constraint parsing to convert free-text queries into structured search filters.
5. **Search Aggregation and Ranking Logic:** Transaction-level records were transformed into company-level intelligence through aggregation of trade volume, shipment count, average price, and activity recency, with ranking logic applied to improve result relevance.
6. **Restricted Product Access and Tokenization:** A token-based access control mechanism was implemented to restrict premium trade intelligence by product subcategory or HS code category, with preview-level visibility for restricted users and unlock functionality for premium access.
7. **Subscription and Monetization Module:** Subscription plans, redeemable codes, token balances, and access grants were integrated into the platform to support a commercially realistic premium intelligence model.
8. **Testing, Validation, and Documentation:** The platform was tested through automated checks, script-based validation, workflow testing, and evaluation of search behavior, followed by preparation of technical documentation and final presentation material.

1.2 Key Challenges

This section discusses the main challenges encountered in the project and the ways in which they were addressed.

One of the major challenges in ZaraiLink was handling noisy and inconsistent trade data. Real-world shipment records often contain spelling variation, incomplete fields, inconsistent company naming, and non-standardized product descriptions. This made direct analytics and search difficult. To address this, a structured data preparation process was developed to clean, normalize, and classify the records before loading them into the application database.

Another important challenge was interpreting highly variable user search behavior. Users may search using exact HS codes, partial product names, informal trade language, or mixed natural language phrases. A search engine built only for strict keyword matching would not have been sufficient. For this reason, the system was designed with both a deterministic fast path and a multi-stage Natural Language Understanding pipeline so that structured and conversational queries could be handled within one unified framework.

A further challenge involved ensuring that premium access restrictions could not be bypassed through alternative query routes. Since the same product intelligence could be reached through HS code searches, category searches, product-name searches, or natural language queries, access control had to remain consistent across all of them. This required backend-level entitlement checking and careful alignment between search results, dashboard views, and token-based unlock logic.

Performance and responsiveness also presented a challenge. Search, aggregation, and query interpretation had to operate on trade datasets large enough to support realistic analytics while remaining responsive for the user. This became especially important in the Natural Language Understanding pipeline, where expensive model inference could increase latency. To manage this, the platform used efficient routing logic, regex-based short-circuiting for simple constraints, and caching support to reduce unnecessary computation.

Another challenge was balancing usability with monetization. The platform needed to provide enough visible value to encourage user engagement while still protecting premium intelligence and contact information behind the token economy. This led to the implementation of preview-based result visibility, controlled unlocking workflows, and clearly separated access states so that restricted and unrestricted experiences remained understandable to the user.

Finally, maintaining coherence across multiple modules was itself a challenge. ZaraiLink combines trade data engineering, intelligent search, company directory functionality, tokenized access, and frontend workflow management within a single platform. Ensuring that these modules worked together reliably required careful separation of concerns, consistent data modeling, and repeated testing across both backend logic and user-facing workflows.

2. Literature Review

2.0.1 Introduction

Information Retrieval (IR) systems form the backbone of modern search engines, enabling efficient discovery of relevant information from large-scale datasets. Traditional search systems relied primarily on lexical matching techniques such as keyword-based retrieval, where relevance was determined by direct overlap between query terms and document text. However, with the increasing complexity and ambiguity of user queries, modern search engines have evolved toward multi-stage retrieval architectures that integrate query understanding, semantic retrieval, and ranking-based optimization strategies.

Recent advancements in information retrieval research highlight a clear transition from purely term-based models toward semantic and hybrid retrieval approaches. Contemporary systems typically adopt a multi-stage pipeline consisting of query processing, candidate retrieval, and ranking or re-ranking phases, allowing them to efficiently scale to millions or even billions of records while maintaining high relevance quality. This shift has been driven by limitations in traditional bag-of-words approaches, particularly their inability to capture semantic similarity and contextual meaning in user queries.

In parallel, advancements in natural language processing and deep learning have significantly enhanced query understanding capabilities within search systems. Techniques such as transformer-based embeddings, zero-shot entity extraction, and few-shot intent classification have enabled systems to better interpret user intent and structure unstructured queries into meaningful representations. These developments have further enabled the integration of semantic search techniques, where vector-based similarity complements traditional lexical retrieval methods.

Despite these advancements, building an effective search system for domain-specific applications such as trade intelligence remains challenging. Such systems require not only accurate retrieval but also structured understanding of entities, intents, and contextual constraints such as trade direction, product categorization,

and transactional behavior. This necessitates a specialized architecture that combines multiple information retrieval paradigms into a unified pipeline.

Motivated by these developments in information retrieval and natural language processing, ZaraiLink is designed as a multi-stage search and intelligence system that integrates query understanding, entity extraction, product resolution, aggregation, and ranking mechanisms. This literature review examines the foundational research behind these components and extracts design principles that inform the development of the ZaraiLink search engine.

2.0.2 Modern Information Retrieval Systems

Modern information retrieval systems are typically designed as multi-stage pipelines to balance two key requirements: efficiency and relevance. Since search systems often operate over extremely large datasets, it is not practical to apply complex ranking models to every document. Instead, the retrieval process is divided into multiple stages, where each stage performs a specific task and progressively refines the results. This multi-stage design has become a standard approach in modern search systems [1].

At a high level, the pipeline begins with query processing, where the user’s input is analyzed to better understand intent and extract meaningful information. This step is important because user queries are often ambiguous or incomplete, and poor understanding at this stage can negatively impact result quality. Following this, the system performs candidate retrieval, where a subset of potentially relevant documents is selected from the full dataset. This stage focuses on efficiency and aims to achieve high recall by ensuring that relevant documents are not discarded early in the process [1].

Traditional retrieval methods rely on keyword-based matching, which works well when the query and documents use similar vocabulary. However, these methods struggle with the vocabulary mismatch problem, where different words are used to express the same meaning. As highlighted in recent research, this limitation often prevents relevant documents from being retrieved in the initial stage, reducing overall system effectiveness [1].

To address this issue, modern systems incorporate semantic retrieval techniques, which allow matching based on meaning rather than exact words. These approaches are generally categorized into sparse and dense retrieval methods. Sparse methods extend traditional keyword-based approaches and are effective for exact matches, especially for structured or technical terms. Dense methods, on the other hand, use vector representations to capture semantic similarity between queries and documents.

However, both approaches have trade-offs, leading to the adoption of hybrid retrieval methods that combine lexical and semantic signals to improve performance [1].

Once a set of candidate results is retrieved, the system applies ranking and re-ranking techniques to order them based on relevance. Unlike the retrieval stage, ranking operates on a much smaller set of documents, allowing the use of more advanced models and additional signals to improve accuracy. This separation between retrieval and ranking is essential for maintaining both scalability and effectiveness in modern IR systems [1].

The concepts discussed in this literature form the basis for the design of the ZaraiLink search engine. In particular, the use of a multi-stage pipeline influenced the system’s overall architecture, with separate components for query understanding, retrieval, and ranking. Additionally, the idea of combining lexical and semantic approaches guided the use of hybrid retrieval to handle both structured trade data and natural language queries effectively.

2.0.3 Query Understanding and Intent Detection

Understanding user intent is a key challenge in modern search systems, especially when queries are written in natural language and may be ambiguous. Traditional approaches based on keyword matching or manually defined rules often fail in such cases, as they rely on exact word overlap and cannot capture the actual meaning behind a query. This becomes a major issue in domain-specific systems, where the same intent can be expressed in many different ways.

Recent work such as SetFit [5] proposes an embedding-based approach to text classification that is both efficient and effective in low-data settings. Instead of relying on large labeled datasets, SetFit uses sentence transformers to convert queries into dense vector representations, allowing the model to capture semantic meaning rather than just keywords. It follows a two-step training process: first, the model is fine-tuned using contrastive learning on sentence pairs, and then a lightweight classification head is trained on top of these embeddings.

One of the key ideas in SetFit is the use of few-shot learning, where the model can adapt to a specific task using only a small number of labeled examples. This is particularly useful in domains like trade intelligence, where collecting large annotated datasets for every possible query type is not practical. Through contrastive learning, the model learns to bring semantically similar queries closer together while pushing different intents apart. This makes it possible to distinguish between queries that may share similar words but have completely different meanings.

In the context of ZaraiLink, this approach was used to build the intent detection

layer of the search engine. A small, curated dataset of trade-related queries was created, covering different ways users express buying and selling intent. The model was then fine-tuned using a contrastive learning setup, allowing it to learn the semantic difference between queries such as “looking for suppliers of rice” and “who is buying rice.” After generating embeddings, a simple classification layer was trained to predict the final intent.

This approach allowed the system to move away from brittle rule-based logic and handle real-world query variation more effectively. It also made it possible to achieve strong performance without requiring large-scale labeled data or heavy computational resources, aligning well with the constraints of building a practical trade intelligence platform.

2.0.4 Entity Extraction and Structured Query Parsing

The GLiNER paper shows that traditional Named Entity Recognition systems are limited by fixed label sets and require large annotated datasets for every new domain. It introduces a zero-shot approach where entity extraction is treated as a matching problem between text spans and user-defined labels in a shared representation space [6]. This removes the need for retraining when entity types change and enables extraction of unseen categories directly from natural language.

In ZaraiLink, GLiNER is used as a zero-shot entity extraction module instead of a supervised NER model. The system provides dynamic labels at runtime such as product, country, quantity, and price. The model extracts relevant spans from raw user queries without requiring domain-specific training data. Queries like “buy 500 tons of sugar from Brazil under 400 dollars” are converted into structured fields for downstream processing.

The GLiNER paper also shows that bidirectional encoders and parallel span classification enable efficient and context-aware extraction. This supports real-time query processing requirements in search systems. In ZaraiLink, extracted entities are used directly in downstream stages. Country entities are mapped to location filters, product spans are used for search matching, and numeric fields such as quantity and price are used for constraint-based filtering.

GLiNER enables structured query formation without predefined schemas or dataset labeling. This design allows ZaraiLink to handle variable and unseen trade queries while maintaining consistent entity extraction across inputs.

2.0.5 Query Representation and Keyword Extraction

Keyword extraction is an important step in information retrieval systems, as it helps convert raw queries into a compact and meaningful representation. Early approaches such as TextRank [3] and term-weighting methods like TF-IDF focus on identifying important words based on frequency or graph structure. While these methods are effective for structured text, they are limited to surface-level signals and do not capture the semantic meaning of phrases.

Recent work shifts toward embedding-based representations, where text is encoded into dense vectors that capture context and meaning. Models such as Sentence-BERT [4] show that semantic similarity can be computed beyond exact word overlap. This is useful in domain-specific systems, where users may express the same intent using different wording.

Based on this, ZaraiLink uses a semantic approach to keyword extraction instead of relying only on frequency-based methods. The goal is to remove filler phrases and retain only the core terms that represent the query. For example, a query such as “I want to find suppliers of dextrose anhydrous” is reduced to “dextrose anhydrous,” while “looking for buyers of Pakistani rice” is reduced to “Pakistani rice.”

To implement this, the system uses KeyBERT [2], which extracts keywords by computing similarity between the full query embedding and candidate phrases. This allows the system to identify terms that are most representative of the query’s meaning rather than just the most frequent words.

The extracted keywords are then used in downstream retrieval, improving matching accuracy while keeping the system robust to variations in query phrasing.

2.1 Existing Trade Intelligence Platforms

2.1.1 Tridge

Tridge is a global trade intelligence and sourcing platform that combines market data, supplier discovery, and transaction support into a unified system. It focuses on transforming large-scale trade data into actionable insights for buyers and suppliers.

Key Capabilities:

- Market intelligence dashboards including price trends, demand signals, and policy updates
- Supplier discovery and sourcing tools

- Integration of internal and external trade data
- Modular product structure spanning analytics, sourcing, and payments

Insights Extracted:

- Trade platforms must combine data, intelligence, and execution rather than only presenting raw data
- Modular system design improves usability by separating analytics, sourcing, and operations
- Large-scale trade data becomes valuable only when converted into actionable insights
- Integration of internal and external data improves decision-making

Application to ZaraiLink: These insights influenced the design of ZaraiLink as a modular trade intelligence system. The platform was structured around unified search and discovery, HS code and product-level intelligence, company-level analysis, and a trade directory for verified partner discovery. This allows users to move from search to analysis to decision-making within one connected workflow.

2.1.2 Volza

Volza is a trade intelligence platform that provides access to large-scale customs and shipment data, enabling users to identify buyers, suppliers, and trade opportunities through detailed filtering and search.

Key Capabilities:

- Global customs and bill-of-lading data search
- Advanced filtering using HS codes, companies, countries, and ports
- Trade flow tracking and competitive intelligence tools
- Exportable reports and datasets

Insights Extracted:

- Verified customs data is a critical foundation for trade intelligence systems
- Search and filtering are central to enabling targeted discovery of trade opportunities

- Structured trade attributes such as HS codes, volumes, and prices are essential for meaningful analysis
- Raw datasets must be easily queryable to support real-world decision-making

Application to ZaraiLink: These insights informed the development of ZaraiLink’s search engine and data layer. The platform emphasizes query-based access to structured trade data, allowing users to search using natural language and filters. This directly influenced the design of the unified search module and trade directory, enabling efficient discovery of buyers, suppliers, and trade flows using structured attributes.

2.1.3 TradeForesight

TradeForesight is a comprehensive trade intelligence platform that integrates global datasets, analytics dashboards, and AI-driven advisory tools to support exporters, importers, and policymakers.

Key Capabilities:

- Integration of multiple global trade databases
- Advanced analytics dashboards and visualizations
- AI-driven insights, forecasting, and matchmaking
- Enterprise and government-focused tools

Insights Extracted:

- Combining multiple data sources enhances coverage and reliability of insights
- Dashboards are essential for summarizing complex trade data into usable insights
- AI-driven analysis enables predictive and decision-support capabilities
- Trade intelligence systems benefit from both historical data and forward-looking insights

Application to ZaraiLink: These insights influenced the analytical components of ZaraiLink. The platform incorporates market intelligence dashboards and company-level intelligence views, enabling users to analyze trends, compare performance, and identify opportunities. This ensures that ZaraiLink goes beyond data retrieval and provides meaningful intelligence for decision-making.

2.1.4 Gap Analysis and Motivation

Existing trade intelligence platforms provide useful features such as data analysis, supplier discovery, and market insights. However, they mostly depend on structured inputs such as filters, dashboards, and fixed queries. This makes them less suitable for users who prefer to search using natural language, where queries can be flexible and unclear.

Another issue is the lack of a connected system. In many platforms, search, analysis, and decision-making are handled in separate modules. Users have to move between these parts, which interrupts the overall workflow and makes the process less efficient.

These platforms are also not built specifically for environments like Pakistan. They are often expensive, globally oriented, and not sufficiently localized for smaller exporters, importers, and trading firms. This makes them harder to use for businesses that need practical trade intelligence without high platform costs or technical barriers.

To address these gaps, ZaraiLink is designed as a unified trade intelligence system built around intelligent search and structured analytics. It combines intent detection, entity extraction, keyword processing, aggregation, ranking, and controlled access into one connected workflow. The platform allows users to move smoothly from search to interpretation, from product-level insight to company-level intelligence, and from discovery to commercial engagement.

3. Software Requirement Specification (SRS)

This chapter provides detailed specifications of the system under development.

3.1 Functional Requirements

This chapter presents the detailed functional and non-functional requirements of the ZaraiLink platform. The system is designed to support trade discovery, analysis, and decision-making through a combination of search, analytics dashboards, and structured company data.

3.1.1 User Authentication

This module covers account creation, session management, and credential recovery.

- The system should allow a new user to register by providing a full name, email address, password, and country.
- The system should validate that the provided email address is not already associated with an existing account and that the password meets a minimum length requirement.
- The system should allow a registered user to log in using their email address and password, and should reject incorrect credentials with an appropriate error message.
- The system should allow an authenticated user to log out, ending their session.
- The system should provide a forgot-password flow: when a user submits their registered email, the system should send a password-reset link to that address.

- The system should allow a user to set a new password by following a valid, one-time reset link.
- The system should redirect authenticated users away from login and signup pages to the main dashboard.
- The system should restrict protected pages (dashboard, trade directory, watchlist, profile, subscriptions) to authenticated users only, redirecting unauthenticated visitors to the login page.

3.1.2 User Profile & Account Management

This module lets users view and maintain their personal account details and security settings.

- The system should display the authenticated user's full name, email address, and account status on the profile page.
- The system should allow a user to edit and save their display name.
- The system should allow a user to change their password by first verifying their current password, then entering and confirming a new password of at least eight characters.
- The system should display the user's current token balance prominently on the profile page.
- The system should show a chronological ledger of all token activity, including credits (top-ups, redeemed codes) and debits (data unlocks), with dates and descriptions.
- The system should show a table of all data entitlements the user has purchased, listing the product or HS code category, the level of access granted, the date of purchase, and the number of tokens spent.
- The system should display summary statistics for the user's token usage, including total tokens spent and their most-accessed trade category.
- The system should provide a direct link from the profile page to the subscription and top-up page.

3.1.3 Dashboard & Landing

This module is the primary entry point after login and the public-facing marketing page.

- The system should present a public landing page to unauthenticated visitors that describes the platform’s capabilities and provides clear entry points to sign up or log in.
- The system should display a central dashboard to authenticated users, serving as the main navigation hub for all platform features.
- The system should present a prominent search bar on the dashboard through which users can initiate a trade search.
- The system should provide navigation links on the dashboard to the Trade Directory, Watchlist, and Subscription sections.

3.1.4 Trade Search Engine

This is the platform’s unified search entry point. All queries — whether a natural language sentence, a product name, or an HS code — are entered in the same search bar. The system routes the user to one of two output pages depending on query type:

- **Search Results page** — produced when the user enters a product name or a natural language query with an import/export scope selected. Returns a ranked list of suppliers or buyers for that specific product.
- **Data Dashboard page** — produced when the user enters a numeric HS code. Returns aggregated market intelligence for the full HS code category.

Both paths share the same autocomplete, paywall, and access-control logic described below.

Input & Query Interpretation

- The system should accept three types of query input from a single search bar: (a) natural language sentences expressing intent (e.g., “I want to buy dextrose from China under \$700”), (b) plain product name searches (e.g., “Dextrose Anhydrous”), and (c) numeric HS code lookups (e.g., “1702”, “1702.3000”).

- The system should automatically detect the query type and display a mode badge to the user (“AI Query Mode”, “Product Search”, or “HS Code Mode”) so they know how their input is being interpreted.
- The system should provide an Import / Export scope toggle on the search home page; selecting a scope and entering a natural language query activates AI Query Mode, which infers the user’s intended trade direction from the query text.
- The system should provide an autocomplete dropdown as the user types at least two characters, suggesting matching product names, their HS code, parent category, and total trade volume, so the user can select the exact product quickly.
- The system should support progressive HS code drill-down in the autocomplete: typing a short HS prefix (e.g., “17”) suggests intermediate chapters with a “Drill Down” indicator; typing or selecting a more specific code narrows to leaf-level products.
- The system should present a disambiguation panel when a typed product name matches multiple distinct variants, allowing the user to select the exact variant before results are displayed.
- The system should provide intent pills (“I want to buy”, “I want to sell”, “Find suppliers”, “Find buyers”) that pre-fill the search input to guide less experienced users.

Search Results Page

- The system should allow the user to switch between four trade perspectives on the results page — Foreign Suppliers, Foreign Buyers, Pakistani Buyers, and Pakistani Suppliers — using tab-style controls, without re-entering the query.
- The system should display a market snapshot alongside results, showing the total number of matching entities, a volume-weighted average price per metric ton, and the leading country of origin.
- The system should display a result card for each entity showing: company name, country, average price per metric ton, total trade volume, number of shipments, and last shipment date.

- The system should allow users to filter results by country of origin, minimum and maximum price, and minimum volume, and to sort by relevance, price, or volume.
- The system should notify the user and suggest switching trade direction when the searched product has no records in the selected direction but does have records in the opposite direction.
- The system should notify the user when a query is too broad to return meaningful results and prompt them to refine it.
- The system should allow a user to select up to four entities from results and navigate to a side-by-side comparison view.

Data Dashboard Page (HS Code Path)

- The system should display a full data dashboard for any HS code entered, showing the total shipment count, a sidebar listing product subcategories within that HS code with record counts, and a ranked list of suppliers or buyers.
- The system should allow the user to switch between the four trade perspectives on the data dashboard using tab-style intent controls.
- The system should allow the user to filter the data dashboard by one or more product subcategories within the selected HS code.
- When an HS code prefix is entered that has child codes rather than leaf-level products, the system should display a navigation page listing the child categories, their descriptions, and their record counts, allowing the user to drill down further.

Paywall & Access Control

- The system should enforce a paywall on all search and dashboard results: users without an access grant for a given product or HS code category should see a preview limited to two result cards, with the remaining results locked.
- The system should display the total number of results found even when the user's access is restricted to the preview, so they can make an informed decision about whether to purchase access.

- The system should allow a logged-in user to unlock full results for a product subcategory or a full HS code category by spending the required number of tokens from their balance.

3.1.5 Deal Detail — Supplier & Buyer Intelligence

This module provides a deep, product-scoped intelligence report for a single supplier or buyer selected from the Search Results page. It is organised as four named tabs.

Overview Tab

- The system should display a summary of the entity’s activity for the searched product, including the active trading period, last active date, typical shipment size, primary sourcing or destination region, and shipment frequency pattern.
- The system should display the entity’s price positioning, showing whether their pricing is above or below the market average for that product, along with a price stability assessment.
- The system should display a miniature price movement chart for the entity’s historical pricing.
- The system should display a shipment-size distribution chart showing the percentage of orders in small, medium, and large size buckets.
- The system should display a list of the entity’s top counterparties (buyers or suppliers) by volume for the searched product.
- The system should display a geographic presence breakdown showing the top countries the entity ships to or sources from, with volume and percentage bars.

Market & Pricing Tab

- The system should display a market overview panel for the searched product, showing total market volume, total market transactions, dominant trade direction, and the top trade route.
- The system should display a monthly market price trend line chart, along with minimum, maximum, and median price statistics for the product category.

- The system should display the entity's price positioning relative to the market average, showing the percentage differential and whether the entity is priced above or below market.
- The system should display a country-level pricing table showing the average price per metric ton and volume share for each country of origin in the market.
- The system should display a supply chain flow table listing the top trade routes (source country to destination country) with volume, market share, and average price.
- The system should display a demand and volume trends section with a monthly bar chart and summary indicators for trend direction, peak demand period, and low demand period.
- The system should display a competitive benchmarking table ranking all suppliers or buyers in the category by volume, with each entity's average price and its percentage deviation from the market average; the currently viewed entity should be highlighted in the ranking.

Company Tab

- The system should display an all-products company overview showing total volume traded across all products, total transaction count, estimated trade value in USD, and the company's import/export direction split.
- The system should display the company's product portfolio with a ranked list and pie chart of top products by volume, including each product's share percentage and average price.
- The system should display the company's partner network, listing top buyers or suppliers across all products with volume and share percentage, and a relationship concentration analysis.

Transactions Tab

- The system should display a paginated table of individual shipment records for the selected entity scoped to the searched product, showing shipment date, product description, buyer, seller, origin country, destination country, quantity in metric tons, and price per metric ton.

- The system should allow the user to filter the transactions table by date range, counterparty name, country of origin or destination, and quantity or price thresholds.
- The system should allow the user to download the displayed transactions as a PDF report.

3.1.6 Supplier & Buyer Comparison

This module lets users compare up to four suppliers or buyers side by side. It is accessed by selecting entities on the Search Results page.

- The system should display a structured comparison table for up to four selected entities, with each entity occupying a column and trade metrics occupying rows.
- The system should include the following metrics in the comparison: average price per metric ton, total volume traded, number of shipments, date last active, historical price range (minimum to maximum), and the countries the entity ships to or buys from.
- The system should visually highlight the best-performing entity in each metric row (e.g., lowest price, highest volume, most recent activity) so the user can instantly identify the strongest option.
- The system should display a price trend chart for each entity, showing how their price per metric ton has changed over the past year.
- The system should allow the user to download the comparison as a formatted PDF report, including the comparison table and historical price data for each entity.
- The system should provide a link from each column of the comparison table to that entity's full deal detail page.

3.1.7 Trade Directory

This module is a searchable, filterable directory of verified companies operating in Pakistan's agricultural trade sector.

- The system should allow a user to browse a directory of verified suppliers and a separate directory of verified buyers.

- The system should allow a user to search the directory by company name.
- The system should allow a user to filter directory listings by country and sector.
- The system should allow a user to sort directory results alphabetically by company name.
- The system should display a paginated grid of company cards, each showing the company name, country, sector, and company type.
- The system should allow a user to add any company card to their watchlist directly from the directory listing.

3.1.8 Company Profile

This module shows the detailed profile of a single verified company in the Trade Directory.

- The system should display a company profile page showing the company's name, legal name, location (district, province, country), sector, role, company type, year established, number of employees, and website.
- The system should display the products the company deals in, including variety and value-added certifications where available.
- The system should display a list of key contacts associated with the company, showing each contact's name, designation, and seniority category.
- The system should mask the contact details (phone, email, WhatsApp) for contacts that have not been unlocked, showing only a blurred preview so the user can see that the information exists.
- The system should allow a logged-in user to unlock a contact's details by spending one token from their balance; once unlocked, the full phone number, email, and WhatsApp number should be revealed.
- The system should notify the user if they have insufficient tokens to unlock a contact and provide a direct link to purchase more tokens.
- The system should show a verification badge on companies that have been confirmed as verified trading partners.

3.1.9 Watchlist

This module lets users save and track companies they are interested in.

- The system should allow an authenticated user to save any company to a personal watchlist by clicking a bookmark icon.
- The system should display a dedicated watchlist page listing all saved companies, showing the company name and the date it was added.
- The system should allow the user to search within their watchlist by company name.
- The system should allow the user to sort watchlist items by date added (newest or oldest first) or alphabetically by name.
- The system should allow the user to remove individual companies from their watchlist.
- The system should allow the user to clear their entire watchlist in a single action, with a confirmation prompt.
- The system should allow the user to export their watchlist to a downloadable file containing company names and dates added.
- The system should allow the user to click on any watchlist item to navigate directly to that company's full profile page.

3.1.10 Token Wallet & Subscription Plans

This module handles the platform's monetisation layer: subscription plans, token balances, and data access grants.

- The system should display available subscription plans to the user, each showing a plan name, price, currency, number of tokens included, and a description of features.
- The system should allow a user to activate a subscription plan, which credits the corresponding number of tokens to their account balance.
- The system should allow a user to spend tokens to purchase access to a full HS code category (unlocking all products within it) or to a specific product subcategory.

- The system should prevent a user from purchasing access they already hold for a given HS code or subcategory, and should silently honour their existing access if they attempt to do so.
- The system should deduct the correct number of tokens from the user's balance when access is purchased, and should refuse the transaction if the balance is insufficient.
- The system should immediately reflect the newly purchased access state in search results and dashboard views without requiring a page reload.
- The system should record every token debit and credit in the user's token history, accessible from the profile page.

3.2 Non-Functional Requirements

3.2.1 Performance

- The system should return search results for any product query within 3 seconds under normal load conditions.
- The system should serve autocomplete suggestions within 500 milliseconds of a user typing at least two characters.
- The system should load the data dashboard for a given HS code within 4 seconds.
- The system should render company deal detail pages, including transaction history, within 3 seconds.
- The system should serve static pages (landing page, login, signup) within 1 second.
- The system should apply search result filters and re-sort results within 500 milliseconds without a server round-trip.
- Search service models and indexes should be loaded into memory once at application startup so that no query incurs a cold-start penalty.

3.2.2 Scalability

- The system should be capable of handling at least 100 concurrent active users without degradation in response times.
- The search and data APIs should be stateless so that additional server instances can be added horizontally to increase capacity.
- The database should support the full volume of historical trade transaction records currently loaded (several hundred thousand rows), with query performance maintained through indexed fields on HS code, buyer, seller, trade type, and reporting date.
- The system should use server-side pagination when returning large result sets (transaction tables, directory listings) so that no single response transfers an unbounded number of records.

3.2.3 Security

- The system should never expose full supplier, buyer, or contact details to unauthenticated requests; all protected data endpoints should verify the caller's session before returning any data.
- The system should enforce access restrictions server-side for paywall-protected data; the backend is the authoritative source of what data a given user may see, and no full dataset should be embedded in any API response from which it could be extracted without authorisation.
- The system should store user passwords as salted, cryptographic hashes and should never store or transmit them in plain text.
- The system should use session-based authentication with CSRF protection for all state-changing operations.
- The system should prevent a user from purchasing the same data access grant twice by enforcing a unique constraint on access records.
- Password reset tokens should expire within 24 hours and become invalid after first use.
- Email verification tokens should expire within 24 hours.

- The system should apply the principle of least privilege: public pages and search results endpoints are accessible without authentication, but personal data, contact details, and paywall-protected trade data require authentication and appropriate token grants.

3.2.4 Usability

- The system should provide clear loading indicators whenever data is being fetched, so users are never left looking at a blank or stale screen.
- The system should display informative, plain-language error messages when a search returns no results, when filters exclude all records, or when a network request fails, along with a suggested corrective action.
- The system should indicate to the user exactly how many total results exist for a query, even when their access level restricts them to seeing only a preview, so they can make an informed decision about whether to purchase full access.
- The system should indicate clearly which contacts are locked versus unlocked on a company profile, showing masked previews to communicate value before the user spends tokens.
- The system should confirm any token-spending action (data unlock, contact unlock) with a confirmation dialogue before deducting tokens, and should show the cost and the user's remaining balance.
- The system should be navigable by keyboard and follow logical tab order on interactive elements.
- The system should be responsive and usable on desktop screen sizes with a minimum viewport width of 768 pixels.
- The system should preserve the user's search context (query, scope, selected variant) across page navigation within the search flow, so navigating to a deal detail and pressing back returns them to the correct search results.

3.2.5 Reliability

- The system should target an uptime of at least 99% during business hours.

- The system should degrade gracefully: if the AI-assisted search returns an error, the system should fall back to keyword-based text search without exposing the failure to the user.
- The system should handle cases where trade records contain missing or null values (country, price, volume) without crashing, substituting safe defaults or omitting those fields from display.
- Database write operations that modify a user's token balance or create access grants should be executed atomically, so that a failure at any step does not leave the balance partially decremented or an access grant partially created.
- The system should log server-side errors with sufficient context (query parameters, user state, stack trace) to allow diagnosis and resolution without requiring the user to reproduce the issue.

3.3 Use Cases

This section presents the main use case diagrams for the ZaraiLink platform. The system has two key operational actors relevant to the project scope: the Registered Member and the Platform Administrator. Each actor interacts with the system in a different way depending on their responsibilities and level of access.

3.3.1 Registered Member

The primary actor for this use case is the Registered Member. As shown in Figure 3.1, the registered member interacts with the platform for account access, trade search, data unlocking, company discovery, watchlist management, and subscription handling. Core functionalities include registering and logging into the system, managing profile information, searching trade data using product names or HS codes, viewing search results and dashboards, purchasing access using tokens, reviewing deal details, comparing suppliers or buyers, browsing the trade directory, unlocking contact details, managing a watchlist, and activating subscription-related token features. These use cases represent the main user-facing functionality of ZaraiLink.

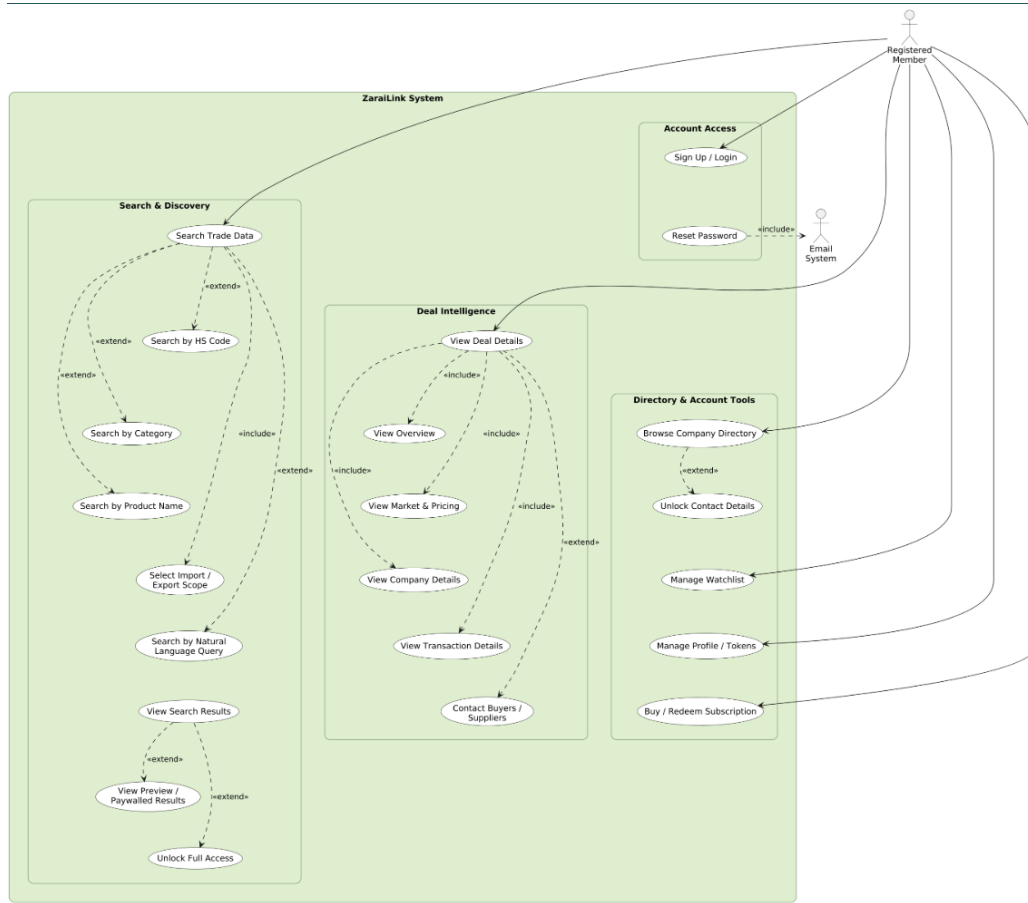


Figure 3.1: Use Case Diagram for Registered Member

3.3.2 Platform Administrator

The primary actor for this use case is the Platform Administrator. As shown in Figure 3.2, the administrator is responsible for managing the operational and data-maintenance side of the platform. Major functionalities include managing user accounts, maintaining company profiles, handling directory taxonomy such as sectors and roles, managing key contacts, administering subscription plans, generating promotional redemption codes, managing trade data, maintaining the HS code structure, and viewing or revoking user access grants. These use cases ensure that the platform remains accurate, secure, and functionally up to date.

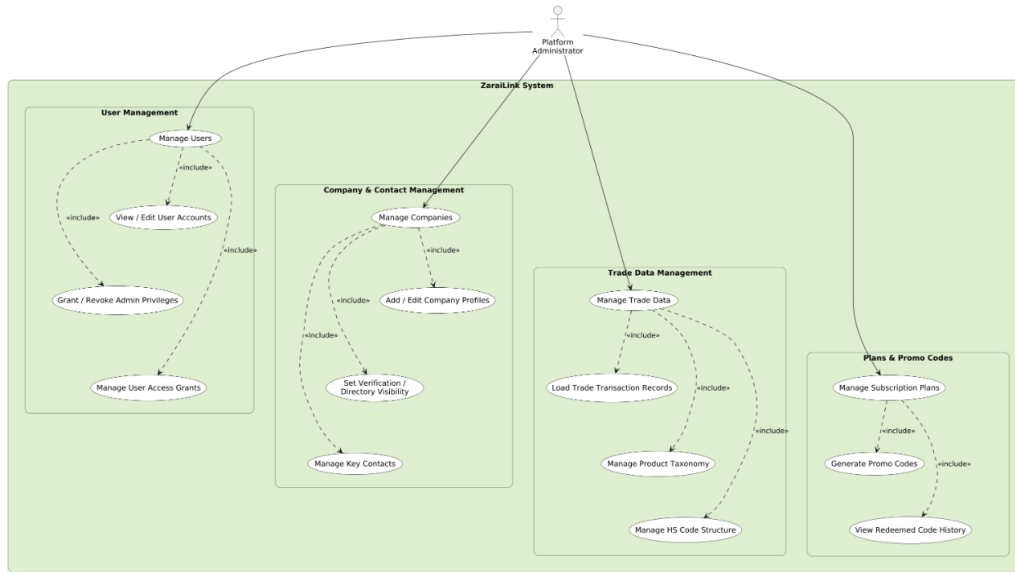


Figure 3.2: Use Case Diagram for Platform Administrator

3.4 User Interfaces

This section presents the main user interfaces of the ZaraiLink platform and highlights the core functionality implemented in the system. The selected screens focus on the primary user journey, beginning from the dashboard and search flow, then moving to results, detailed intelligence views, and the trade directory interfaces.

3.4.1 Dashboard

The dashboard serves as the main entry point for authenticated users. It provides a unified search interface through which users can query the platform using different input types, including HS codes, product names, and natural language trade queries. It also provides quick access to the main platform modules such as trade intelligence, subscriptions, and the watchlist.

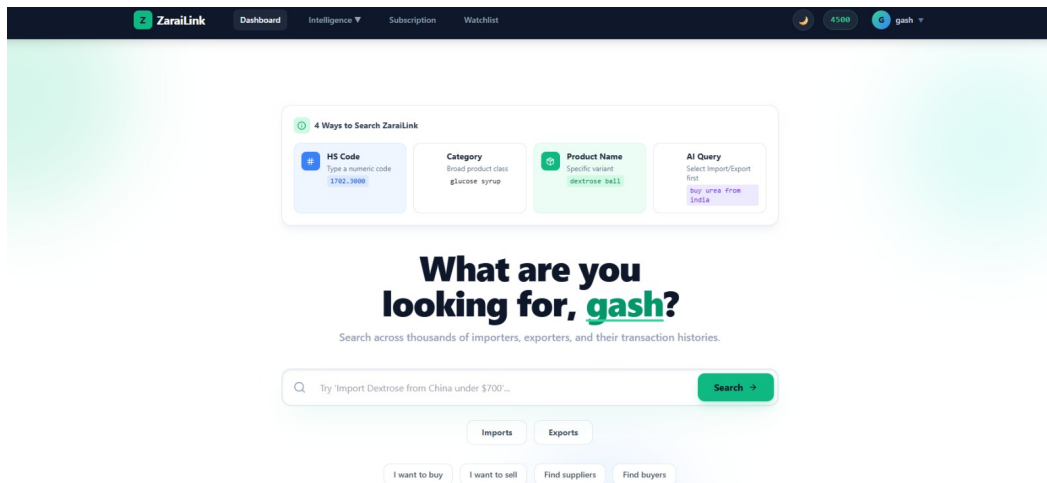


Figure 3.3: Dashboard interface of ZaraiLink

3.4.2 HS Code Results Dashboard

When a user searches using a numeric HS code, the system routes them to the HS code data dashboard. This interface presents aggregated market intelligence for the selected code and allows the user to switch between four trade perspectives: Foreign Suppliers, Foreign Buyers, Pakistani Buyers, and Pakistani Suppliers. Since an HS code alone does not indicate trade intent, the system provides all four tabs so the user can manually select the desired perspective.

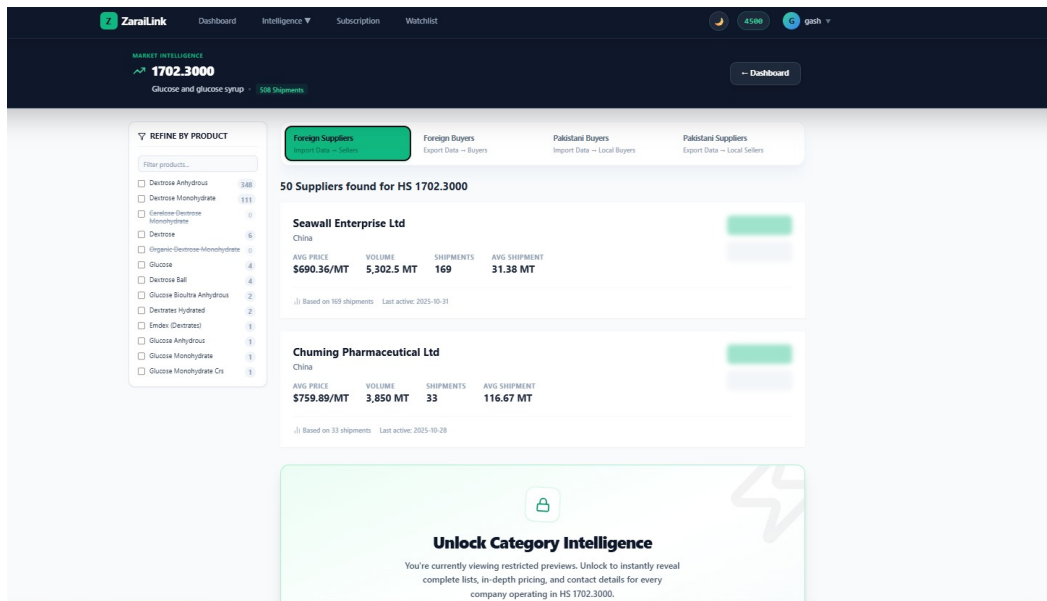


Figure 3.4: HS code results dashboard with four trade perspective tabs

3.4.3 Natural Language Query Results

When a user enters a natural language query together with a selected scope such as import or export, the system interprets the intended trade direction automatically and displays a ranked results page. In this case, the interface does not require four separate trade-perspective tabs because the user intent has already been inferred from the query. The results page shows relevant companies together with market indicators and filtering options.

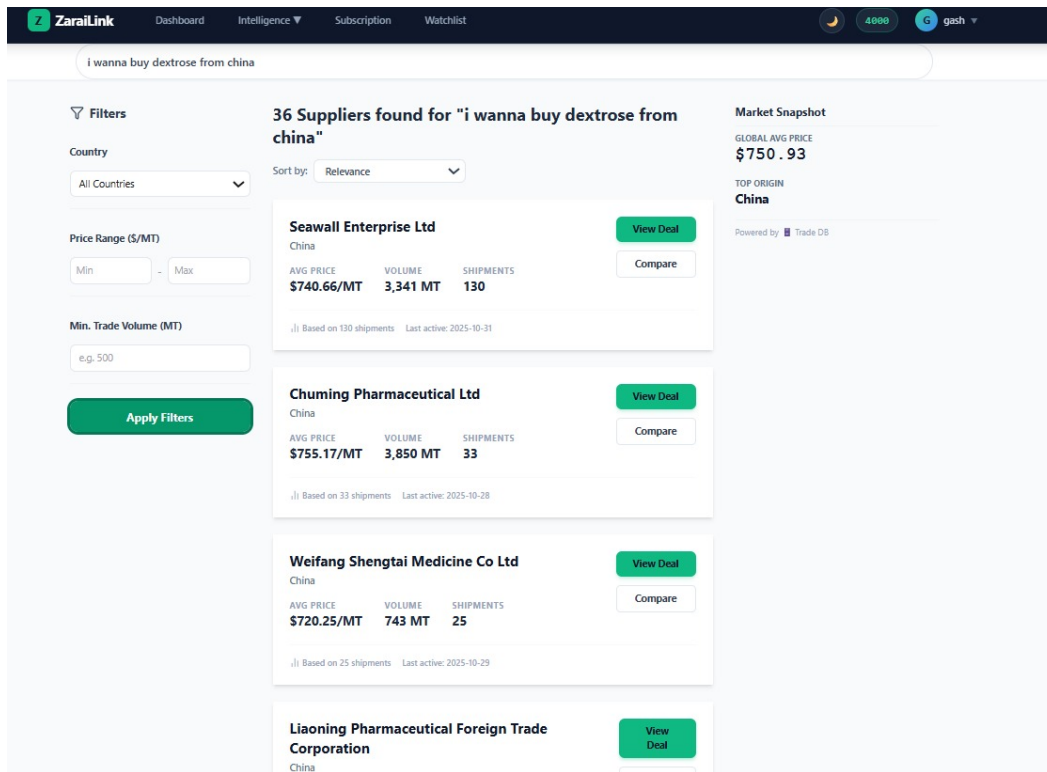


Figure 3.5: Search results for a natural language trade query

3.4.4 Deal Detail Interface

The deal detail interface provides an in-depth intelligence view for a selected supplier or buyer. It presents high-level metrics such as volume traded, number of shipments, average price, and buyer concentration, along with tab-based analysis including overview, market and pricing, company information, and transactions. This page is designed to support deeper decision-making after a result has been selected.

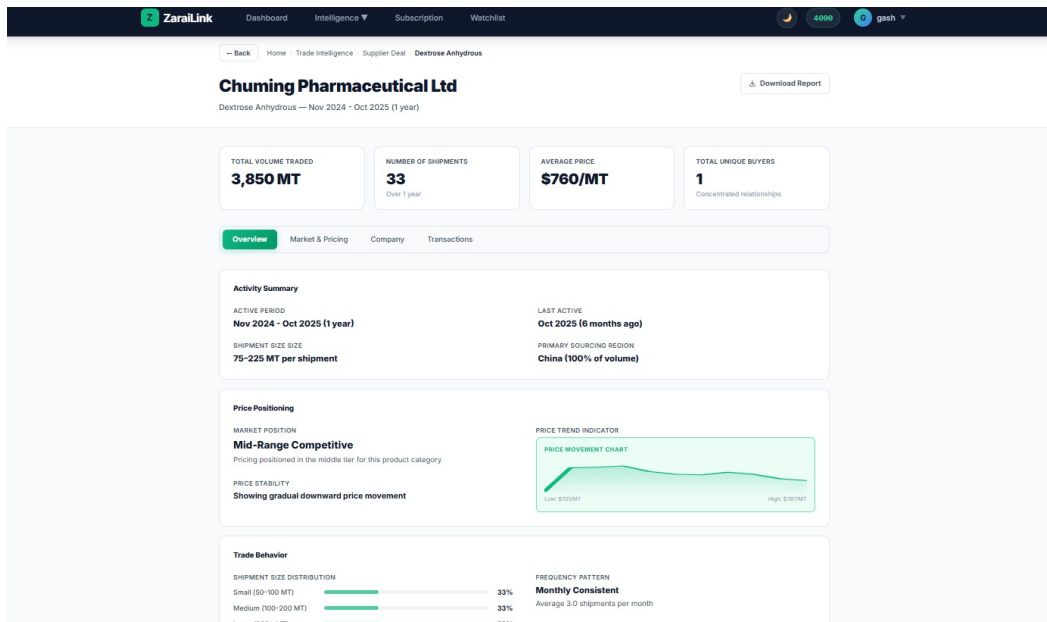


Figure 3.6: Deal detail page for an individual trading entity

3.4.5 Find Suppliers Directory

The Find Suppliers interface is part of the trade directory module. It allows users to browse verified suppliers, search by company name, and explore company cards showing summary information such as country, product category, and business role. The interface also provides direct navigation to detailed company profiles and contact information.

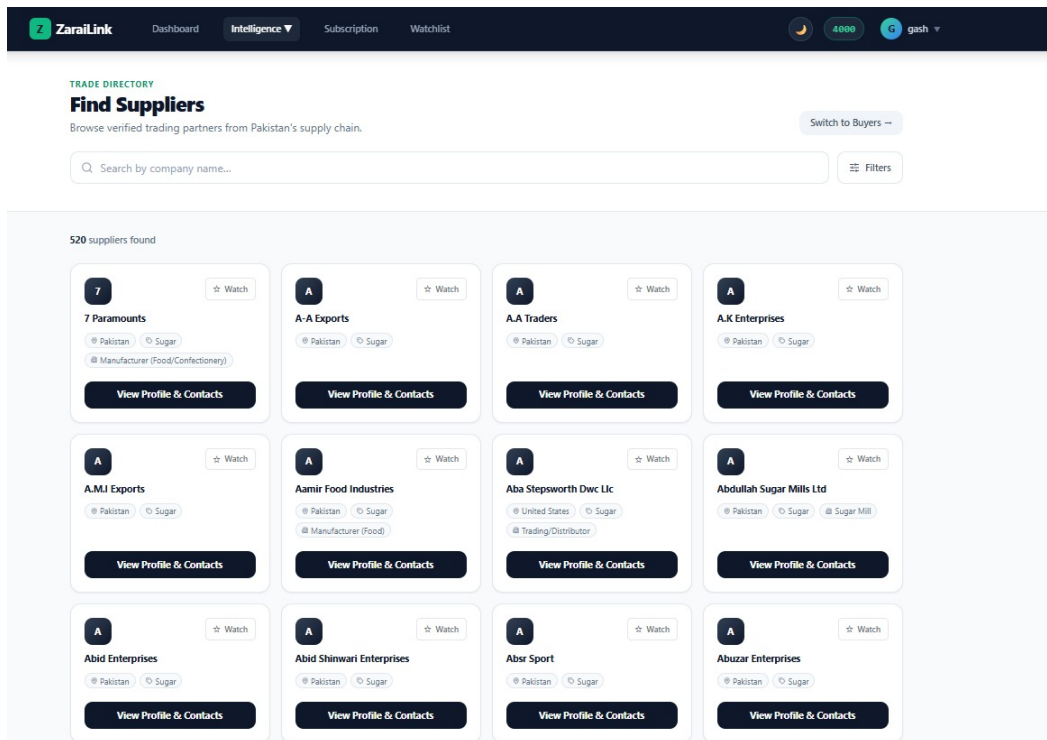


Figure 3.7: Find Suppliers directory interface

3.4.6 Find Buyers Directory

The Find Buyers interface provides a similar directory experience focused on verified buyers and distributors. Users can search and browse buyer profiles, review summary company information, and move to detailed company and contact views. Together with the supplier directory, this interface supports broader trade discovery beyond direct search queries.

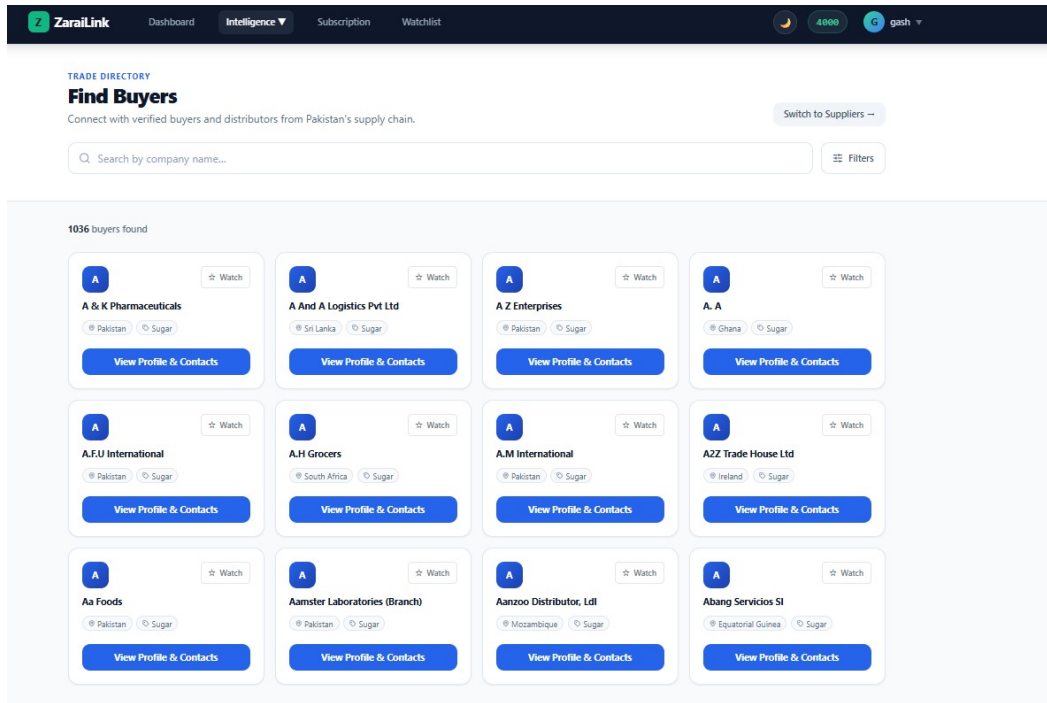


Figure 3.8: Find Buyers directory interface

3.4.7 System Diagram

The diagram in Figure 3.9 provides a high-level overview of the ZaraiLink Trade Intelligence Platform Architecture. It illustrates the major components, the data flow between them, and the technologies and APIs employed. At the center of the system lies the Django Application Server, which acts as the core backend hub handling all incoming API requests from the React frontend. It routes these requests to the appropriate internal services: the NLU Inference Service or the OpenSearch Node, depending on the nature of the request.

The components are described as follows:

- **React Frontend:** The user-facing application built using React (v19) with TailwindCSS and Framer Motion communicates with the backend via asynchronous HTTP requests using the Axios library. It provides modules for trade search, supplier and buyer discovery, HS code analysis, token-based subscriptions, and user account management.
- **Django Application Server:** The core backend built with Django REST

Framework handles all business logic, user authentication, database ORM operations, and orchestration of internal services. It exposes a RESTful API under the `api/<module>/` structure and manages session-based authentication with CSRF and CORS protection.

- **NLU Inference Service:** Embedded within the Django environment under `backend/search/services/`, this service runs the full ML pipeline for natural language query interpretation. It processes user trade queries through SetFit for intent classification, GLiNER for entity recognition, KeyBERT for keyword extraction, RapidFuzz for country normalization, and a Regex engine for numeric constraint parsing.
- **PostgreSQL:** The primary relational database storing users, company profiles, trade shipment records, product taxonomies, and subscription ledgers. Key models include `accounts.User`, `trade_data.Transaction`, `companies.Company`, and `subscriptions.UserAccess`.
- **OpenSearch:** A dedicated search and indexing node deployed via Docker. It stores indexed trade records and supports millisecond-latency full-text and vector search for the Data Dashboard and Trade Directory modules.
- **Redis:** Used for session management and caching of repeated query results, reducing latency for frequently accessed trade data.

External APIs

- **OpenRouter API** — OpenRouter is used within the NLU Inference Service (`nlu_engine.py`) as a fallback LLM gateway routing to models such as DeepSeek and Mistral. It handles complex trade queries that cannot be resolved by the local ML pipeline alone, particularly for price and quantity extraction from ambiguous natural language input.
- **Gmail SMTP** — Gmail's SMTP service is configured in the Django settings for system-triggered transactional emails, including account verification and password reset communications sent to registered users.
- **OpenAI API (Optional)** — Configured as a secondary LLM source for data enrichment tasks, providing an alternative inference backend to OpenRouter where applicable.

This architecture ensures modularity and scalability. The system supports containerized deployment using Docker, with a `docker-compose.yml` file orchestrating the OpenSearch and OpenSearch Dashboards nodes. The clear separation between the NLU service, the search index, and the relational database allows independent updates to the ML models without disrupting core platform functionality.

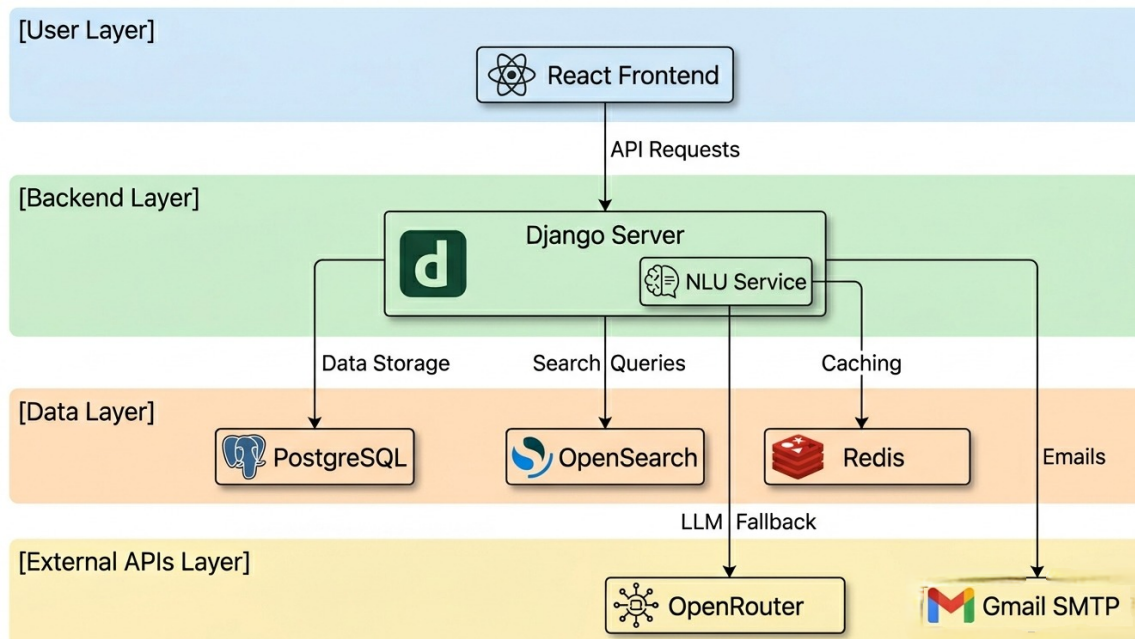


Figure 3.9: ZaraiLink System Architecture Diagram

4. Software Design Specification (SDS)

This chapter provides important artifacts related to design of our project.

4.1 Software Design

This section presents the UML class diagram and gives a brief description of each class in the ZaraiLink system. Attributes and methods of each class, and the relationships among classes, are clearly presented. The following UML diagram (Figure 4.1) illustrates the key components of the system, their attributes, methods, and relationships.

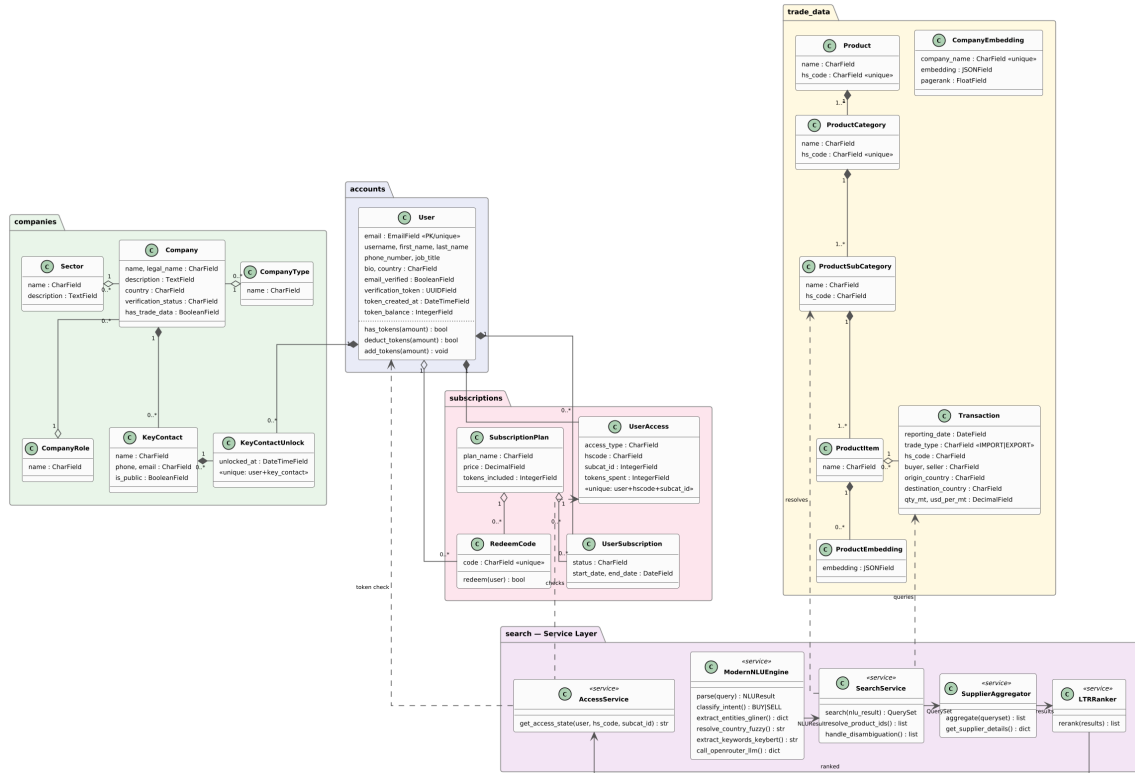


Figure 4.1: UML Class Diagram for ZaraiLink

Detailed Description of the UML Diagram

1. User

- The **User** class is the central entity of the ZaraiLink system and extends Django's **AbstractUser**.
- Key attributes include:
 - **email** (EmailField, unique) as the primary identifier used for authentication,
 - **username**, **first_name**, **last_name**, **phone_number**, **job_title**, **bio**, and **country** for profile information,
 - **email_verified** (BooleanField) to enforce email verification,
 - **verification_token** (UUIDField) for secure verification links,
 - **token_created_at** (DateTimeField) to enforce 24-hour token expiry,

- `token_balance` (`IntegerField`) representing the user’s spendable balance.
- Key methods include:
 - `has_tokens(amount)` to check token sufficiency,
 - `deduct_tokens(amount)` for atomic token deduction,
 - `add_tokens(amount)` to credit tokens after purchases or redemptions,
 - `is_verification_token_valid()` to verify token validity.

2. **SubscriptionPlan**

- The `SubscriptionPlan` class defines the available billing tiers on the platform.
- Its main attributes are:
 - `plan_name` (`CharField`),
 - `price` (`DecimalField`),
 - `tokens_included` (`IntegerField`).
- It is referenced by both `UserSubscription` and `RedeemCode`.

3. **UserSubscription**

- The `UserSubscription` class records active or historical subscriptions.
- Its attributes include:
 - `status` (`CharField`),
 - `start_date` (`DateField`),
 - `end_date` (`DateField`).
- It holds foreign key relationships to both `User` and `SubscriptionPlan`.

4. **RedeemCode**

- The `RedeemCode` class represents voucher codes that can be exchanged for tokens.
- Its attributes include:
 - `code` (`CharField`, `unique`),
 - a foreign key to `SubscriptionPlan`,

- a nullable foreign key to `User` through `redeemed_by`.
- The `redeem(user)` method marks the code as used and credits the user's token balance.

5. `UserAccess`

- The `UserAccess` class is the core of paywall enforcement.
- It records which HS codes or product subcategories a user has unlocked.
- Its attributes include:
 - `access_type` (`CharField`, either `HS_CODE` or `PRODUCT`),
 - `hscode` (`CharField`),
 - `subcat_id` (`IntegerField`),
 - `tokens_spent` (`IntegerField`).
- A unique constraint on (`user`, `hscode`, `subcat_id`) prevents duplicate charges.
- It holds a foreign key to `User`.

6. `Company`

- The `Company` class represents a business entity listed in the Trade Directory.
- Its attributes include:
 - `name` and `legal_name` (`CharFields`),
 - `description` (`TextField`),
 - `country` (`CharField`),
 - `verification_status` (`CharField`),
 - `has_trade_data` (`BooleanField`).
- It holds foreign keys to `Sector`, `CompanyRole`, and `CompanyType`.

7. `Sector`

- The `Sector` class represents industry classification tags.
- Its attributes include:
 - `name` (`CharField`),

- `description` (TextField).
- It holds a one-to-many relationship with `Company`.

8. `CompanyRole`

- The `CompanyRole` class classifies a company as a supplier or buyer.
- Its primary attribute is `name` (CharField).
- It holds a one-to-many relationship with `Company`.

9. `CompanyType`

- The `CompanyType` class defines the legal structure of a company.
- Its primary attribute is `name` (CharField).
- It holds a one-to-many relationship with `Company`.

10. `KeyContact`

- The `KeyContact` class stores contact persons associated with a company.
- Its attributes include:
 - `name` (CharField),
 - `phone` and `email` (CharFields),
 - `is_public` (BooleanField).
- It holds a foreign key to `Company`.

11. `KeyContactUnlock`

- The `KeyContactUnlock` class records which users have unlocked a specific `KeyContact`.
- Its key attribute is `unlocked_at` (DateTimeField).
- It holds foreign keys to both `User` and `KeyContact`.
- A unique constraint on the pair prevents duplicate charges.

12. `Product`

- The `Product` class represents the top level of the HS hierarchy.

- Its attributes include:
 - `name` (CharField),
 - `hs_code` (CharField, unique).
- It is the root of the four-level classification tree.

13. **ProductCategory**

- The `ProductCategory` class represents the 4-digit HS category level.
- Its attributes include:
 - `name` (CharField),
 - `hs_code` (CharField, unique).
- It holds a foreign key to `Product` and is the parent of `ProductSubCategory`.

14. **ProductSubCategory**

- The `ProductSubCategory` class represents the 6–8 digit HS subcategory level.
- Its attributes include:
 - `name` (CharField),
 - `hs_code` (CharField).
- It holds a foreign key to `ProductCategory` and is the parent of `ProductItem`.

15. **ProductItem**

- The `ProductItem` class represents a specific product variant within a subcategory.
- Its main attribute is `name` (CharField).
- It holds a foreign key to `ProductSubCategory`.
- It is referenced by both `Transaction` and `ProductEmbedding`.

16. **Transaction**

- The `Transaction` class stores raw import and export records.
- Its attributes include:
 - `reporting_date` (DateField),

- `trade_type` (CharField),
- `hs_code` (CharField),
- `buyer` and `seller` (CharFields),
- `origin_country` and `destination_country` (CharFields),
- `qty_mt` (DecimalField),
- `usd_per_mt` (DecimalField).
- It holds a foreign key to `ProductItem`.
- It is indexed on `buyer`, `seller`, `hs_code`, and `trade_type`.

17. `CompanyEmbedding`

- The `CompanyEmbedding` class stores vector representations of companies.
- Its attributes include:
 - `company_name` (CharField, unique),
 - `embedding` (JSONField),
 - `pagerank` (FloatField).
- It is queried by the `SearchService` during vector-based retrieval.

18. `ProductEmbedding`

- The `ProductEmbedding` class stores vector representations of product items.
- Its main attribute is `embedding` (JSONField).
- It holds a foreign key to `ProductItem`.

19. `ModernNLUEngine` (*Service*)

- The `ModernNLUEngine` class is the entry point of the NLU search pipeline.
- Its main method is `parse(query)`.
- Important internal responsibilities include:
 - `classify_intent()` using `SetFit`,
 - `extract_entities_gliner()` using `GLiNER`,
 - `resolve_country_fuzzy()` using `RapidFuzz`,
 - `extract_keywords_keybert()` using `KeyBERT`,

- `call_openrouter_llm()` for ambiguous price and quantity extraction.

20. **SearchService** (*Service*)

- The **SearchService** class translates NLU output into database queries.
- Its main method is `search(nlu_result)`.
- Supporting methods include:
 - `resolve_product_ids()`,
 - `handle_disambiguation()`.
- It queries the **Transaction** and **ProductSubCategory** models directly.

21. **SupplierAggregator** (*Service*)

- The **SupplierAggregator** class processes the raw **QuerySet** returned by **SearchService**.
- Its `aggregate(queryset)` method groups transactions by buyer or seller.
- Its `get_supplier_details()` method provides a detailed profile for an individual company.
- It computes metrics such as total shipment volume, average unit price, shipment count, and last shipment date.

22. **LTRRanker** (*Service*)

- The **LTRRanker** class applies a LightGBM Learn-to-Rank model.
- Its `rerank(results)` method reorders aggregated supplier profiles by predicted relevance.

23. **AccessService** (*Service*)

- The **AccessService** class enforces the platform’s paywall logic.
- Its main method is `get_access_state(user, hs_code, subcat_id)`.
- It returns one of three access states:
 - `FULL_ACCESS`,
 - `PRODUCT_ACCESS`,
 - `NO_ACCESS`.

- When access is not available, results are scrubbed to preview level visibility.

This architecture ensures a clean separation between data persistence, business logic, and search intelligence, while maintaining flexibility for future enhancements such as additional ML models, expanded HS code coverage, and richer payroll tiers.

4.2 Data Design

This section describes the logical database design of ZaraiLink and presents the Entity Relationship Diagram (ERD) used to model the core data structures of the platform. The database is designed to support user management, trade intelligence search, company directory functionality, and token-based access control.

4.2.1 Overview

The ZaraiLink database is organised around two main domains: the global trade intelligence dataset and the verified trade directory. The trade intelligence side stores raw shipment activity and classifies it using a hierarchical HS-code structure, beginning from broad product groupings and narrowing down to specific product items. Each transaction record captures key trade attributes such as reporting date, trade type, HS code, buyer, seller, and quantity traded. Since real-world customs and shipment data is often inconsistent, these records are not always strictly linked to verified company entities through direct foreign-key relationships. Instead, they are kept flexible and are later associated with directory entities through the platform's search and intelligence mechanisms.

The second major domain is the Trade Directory, which stores verified companies, their sectors, roles, types, products, and key contacts. This part of the schema supports directory browsing, company profiling, and contact unlocking features. In addition, the database includes a monetisation layer that manages token balances, subscription plans, redeemed codes, and access grants. Supporting entities such as audit logs and contact unlock records provide accountability, access tracking, and platform-level security.

4.2.2 Entity Relationship Diagram

Figure 4.2 illustrates the Entity Relationship Diagram of the ZaraiLink platform. It captures the main entities, their attributes, and the relationships between user

accounts, companies, contacts, trade records, product hierarchy, subscriptions, and access control structures.

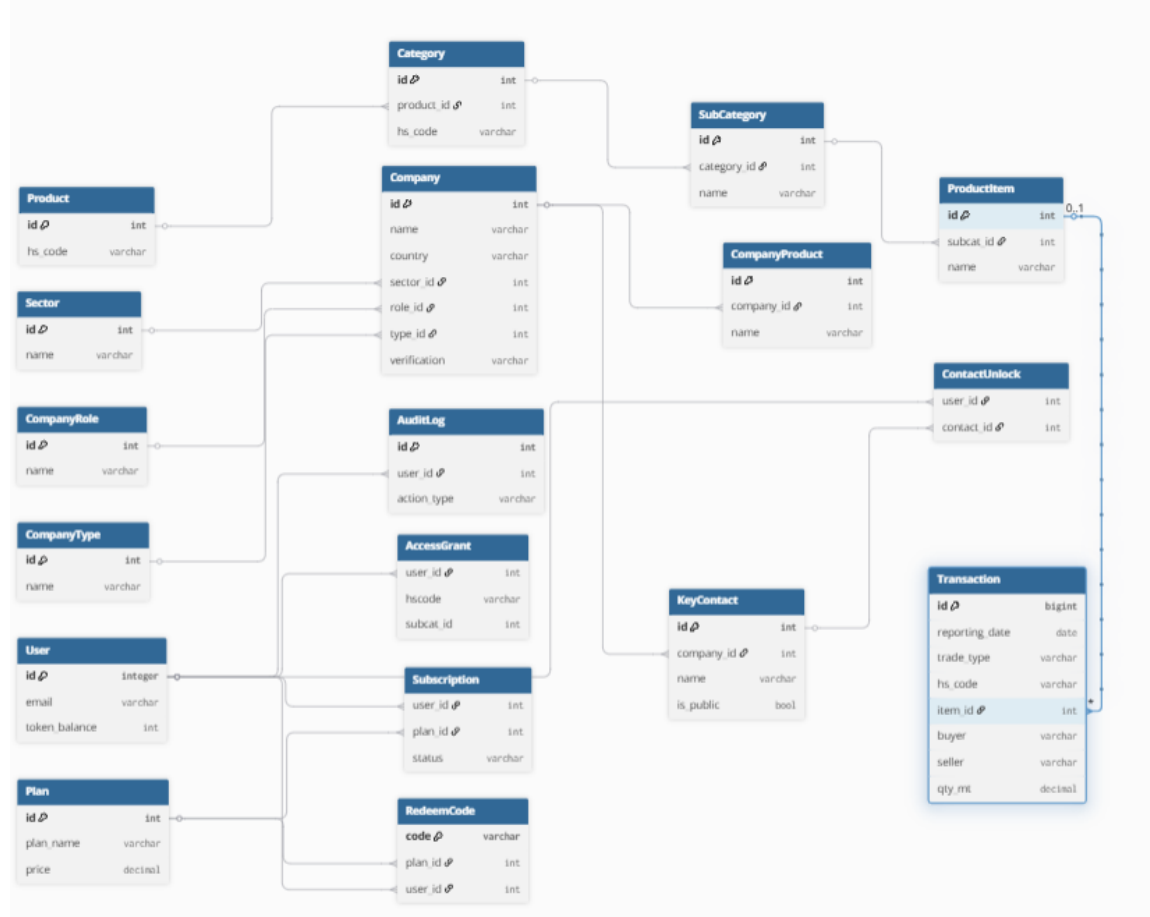


Figure 4.2: Entity Relationship Diagram of ZairiLink

4.2.3 Core Data Groups

The schema can be understood through four main data groups.

User and Audit Data: The **User** entity stores the platform’s registered users, including their account identity and available token balance. The **AuditLog** entity records important user actions for monitoring and accountability.

Trade Directory Data: The directory portion of the schema centres on the **Company** entity, which stores verified supplier and buyer profiles. Supporting entities such as **Sector**, **CompanyRole**, and **CompanyType** classify companies by busi-

ness domain and function. The **CompanyProduct** entity stores product associations, while **KeyContact** stores company contact persons. The **ContactUnlock** relationship records which users have unlocked access to token-gated contacts.

Trade Intelligence Data: The trade intelligence portion models product and shipment information. The hierarchy begins with **Product**, then narrows through **Category**, **SubCategory**, and finally **ProductItem**. The **Transaction** entity stores the shipment-level trade data and references the corresponding product item.

Subscription and Access Data: The monetisation layer is represented through **Plan**, **Subscription**, **AccessGrant**, and **RedeemCode**. These entities manage subscription plans, activated subscriptions, redeemed promotional codes, and the access permissions granted to users for premium trade intelligence.

4.2.4 Design Rationale

The database design separates high-volume trade data from curated directory data so that both can evolve independently while still supporting integrated search and intelligence features. The product hierarchy enables structured drill-down from broad HS-based categories to specific product variants, which is important for autocomplete, search, and dashboard functionality. The monetisation-related entities support ZaraiLink’s token economy by controlling premium data access and contact unlocking in a traceable and scalable way.

The data design provides a structured foundation for the platform’s search engine, directory system, watchlist features, subscription workflow, and protected intelligence access model.

5. Methodology and System Implementation

5.1 Methodology

This section describes the methodology used to design and implement ZaraiLink. The platform was developed as a full stack trade intelligence system that combines structured trade data, natural language query processing, market intelligence generation, and token based access control. The methodology covers the development approach, data preparation process, query handling pipeline, access control mechanism, and frontend interaction flow.

5.1.1 System Development Approach

ZaraiLink was developed using an iterative and modular approach suited to the complexity of a trade intelligence platform. The architecture was designed around a separation of concerns between the presentation layer, the application logic layer, and the data layer. The frontend was implemented as a React based single page application, while the backend was built using Django and REST based APIs. PostgreSQL was used for structured data persistence, and Redis was used to improve the response time of repeated Natural Language Understanding queries.

The backend was organised into domain specific modules:

- **trade_data** manages the product hierarchy and transaction records that power the intelligence engine.
- **search** handles query interpretation, product resolution, aggregation, ranking, and result shaping.
- **accounts** manages authentication, profile handling, and password reset functionality.

- **companies** stores verified company profiles and key contacts.
- **subscriptions** manages token balances, subscription activation, access grants, and premium data enforcement.

Feature implementation followed a vertical integration pattern. Each major feature was developed across both frontend and backend layers before being refined through testing. This included the dashboard search flow, the HS code intelligence dashboard, the trade directory, the watchlist, and the contact unlock mechanism. This development structure improved maintainability and allowed individual modules to evolve without destabilising the rest of the platform.

5.1.2 Data Preparation and Organisation

Data Collection and Consolidation

The data preparation process for ZaraiLink followed a semi automated ETL workflow so that raw trade records could be transformed into a structured dataset suitable for analytics and search. For the implementation and analysis workflow described in this report, approximately 5,000 records were processed into an analysis ready dataset. This included 2,300 import records for sugar covering the years 2024 to 2025, together with 3,000 export records selected from a larger raw export dataset containing approximately 7,500 records.

The first stage focused on extraction and consolidation. Raw source files in spreadsheet and tabular formats were gathered and placed into a staging layer. This staging step preserved the original records before transformation and allowed the data to be prepared for downstream cleaning, mapping, and validation.

Cleaning and Standardisation

The second stage focused on cleaning and normalisation. Automated rules were applied to fix broken rows, remove duplicate and near duplicate records, standardise column names, preserve HS codes in text format, normalise date fields, and convert numeric values into analysis ready forms. Quantities, unit values, and financial amounts were all brought into consistent formats so that later calculations could be performed reliably.

Entity names were also standardised across buyers, sellers, suppliers, shipping agents, and countries. This step was necessary because inconsistent naming would fragment aggregation and reduce search quality. Manual review was used only for

ambiguous cases, exception handling, and business rule validation. This allowed the overall preparation process to remain scalable while still maintaining data quality.

Transformation and Classification

After cleaning, the raw customs style data was transformed into a business oriented reporting structure. Important fields included:

- Date
- HS code
- Category and sub category
- Item description
- Buyer and seller
- Shipping agent
- Country
- Quantity in kilograms and metric tons
- Unit prices and converted value fields

Product categories and sub categories were derived through HS code mapping, description based classification, and standardised taxonomy rules.

Within the application database, the prepared trade data is organised using a hierarchical commodity structure. The hierarchy begins with **Product**, followed by **ProductCategory**, **ProductSubCategory**, and **ProductItem**. This structure allows the platform to support autocomplete, HS code drill down, grouped analytics, and company level aggregation across different levels of commodity specificity.

Validation and Loading

The final stage focused on validation and loading. Quality checks were applied to quantity conversions, value calculations, missing data, and duplicate control. Records that failed automated checks were routed for exception review before being approved for final loading.

The resulting dataset was loaded into the application database and linked to the broader platform data model. In parallel with the trade dataset, the system also

maintains a curated trade directory containing verified companies, sectors, company roles, company types, products, and key contacts. Sensitive contact details are stored separately from public company information so that they can be governed by token based access rules.

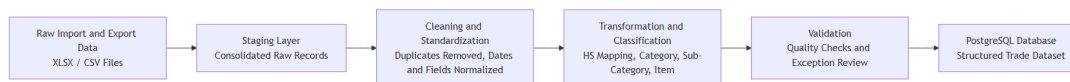


Figure 5.1: Trade data preparation and loading pipeline used in ZaraiLink

5.1.3 Trade Search and Query Processing Pipeline

The ZaraiLink search engine was designed to handle both highly structured trade lookups and flexible conversational queries. To support this, the system follows a dual-track query processing architecture. Queries that are already structured and precise are handled through a deterministic fast-track, while conversational and ambiguous queries are routed through a Natural Language Understanding pipeline. This design improves both performance and resilience by avoiding unnecessary machine learning inference when simpler deterministic logic is sufficient.

Frontend Query Acquisition

The search process begins on the frontend through the dashboard search interface and shared navigation search components. Users can search using:

- an HS code,
- a product or category name, or
- a natural language trade query.

While the user types, a debounced autocomplete mechanism calls the backend autocomplete endpoint after a short delay. This allows the system to suggest canonical product names, matching HS codes, and related trade categories before the search is submitted. This early suggestion step improves precision and reduces ambiguity in user input.

Dual-Track Query Routing

Once a query is submitted, the backend determines whether it should be handled through a deterministic fast-track or through the Natural Language Understanding pipeline.

The deterministic fast-track is used for precise, structured queries. If the query is numerical or already formatted as an HS code, it is routed directly to the HS code dashboard logic. If the user selects a product, category, or subcategory from the autocomplete suggestions, the frontend passes the exact resolved identifier to the backend. In both cases, the query bypasses expensive inference stages and moves directly into database filtering and aggregation. This allows highly specific trade searches to be answered with minimal latency.

The Natural Language Understanding path is used when the input is conversational or contains ambiguity. In these cases, the system must infer trade direction, identify product and country references, and detect any numeric constraints before the query can be mapped onto the trade dataset. After interpretation, all query types converge into a structured representation that is passed to the aggregation layer.

Natural Language Query Interpretation

Natural language queries are interpreted through a multi-stage inference pipeline implemented in the `ModernNLUEngine`. The purpose of this pipeline is to transform unstructured trade language into structured search filters that can be applied to the transaction database.

The interpretation pipeline combines several components:

- SetFit for buy or sell intent classification,
- GLiNER for country, quantity, and price-related entity extraction,
- RapidFuzz for geographic normalization,
- KeyBERT for product keyword extraction,
- regular expressions for simple numeric constraints, and
- OpenRouter fallback for more ambiguous phrasing.

The first stage determines whether the user is attempting to buy or sell. This classification is necessary because the same product query may need to be routed

to either suppliers or buyers depending on trade direction. The next stages identify relevant product references, country mentions, and numeric constraints such as price ceilings or quantity thresholds.

A key design goal of this pipeline is graceful degradation. If one inference component is unavailable or produces weak output, fallback logic ensures that the overall query still remains usable. For example, regex based intent handling can support simple directional phrasing, RapidFuzz can normalize noisy country strings, and stop-word stripping can preserve a workable product phrase when semantic keyword extraction is imperfect. This layered design prevents the search process from failing outright when individual model stages are uncertain or unavailable.

The pipeline also includes a performance-oriented routing rule for constraints. Before invoking a large language model for deeper parsing, the engine first applies tuned regular expressions to identify straightforward numeric conditions such as **under \$500**, **max \$800**, or **minimum 100 metric tons**. If these are successfully parsed, the expensive large language model stage is skipped entirely. This reduces response time and limits unnecessary inference cost for standard business queries.

The final output of the NLU stage is a structured search object containing the inferred intent, product reference, geographic filters, and numeric conditions.

Aggregation and Ranking

After routing and interpretation, the filtered shipment records are passed to the aggregation layer. At this stage, transaction-level records are grouped by trade entity so that the platform can present company-level intelligence rather than a flat list of isolated shipment rows. The aggregation service computes:

- total trade volume,
- average price,
- shipment count, and
- the most recent shipment date.

Ranking is then applied so that the most commercially relevant entities appear near the top of the results. The main ranking signals are volume, frequency of activity, and recency of participation within the queried category. The codebase also includes a conditional Learn-to-Rank component implemented using LightGBM, which can further refine the ordering of top candidates using learned relevance features.

This design allows the same search engine to support exact HS code lookups, canonical product searches, and conversational trade discovery within one coherent architecture. It also ensures that the system remains efficient for structured inputs while still being flexible enough to process noisy or natural language queries.

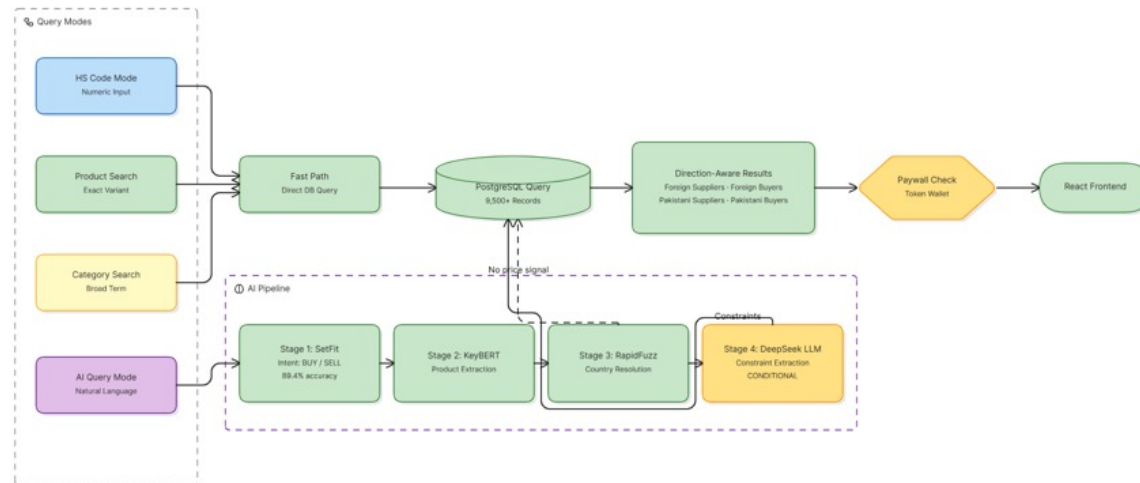


Figure 5.2: Multi query processing pipeline for HS code, product, and natural language search

Access Control and Payload Restriction

Before results are returned to the frontend, the backend checks the user's access state for the requested dataset. This determines whether the user is allowed to view the full intelligence payload or only a restricted preview. Restricted users are shown a limited number of visible result cards, while sensitive data remains unavailable until the proper access grant has been created.

The final response is then rendered either as a ranked search results page or as an HS code intelligence dashboard, depending on the original query type. This design allows one search system to handle structured code lookups, product discovery, and conversational trade intelligence through a unified processing pipeline.

5.1.4 Access Control and Token Enforcement

The premium intelligence features of ZaraiLink are controlled through a token based monetisation model implemented in the `subscriptions` module. The system stores

user token balances, subscription plans, redeemed codes, and access grants representing previously unlocked content. Access may be granted at the level of an HS code category or at the level of a more specific product subcategory.

When a user requests protected data, the backend performs an authoritative access check before constructing the final response. The decision flow is based on three core steps:

- check whether the required access grant already exists,
- return a restricted preview if access is missing, and
- unlock the content after token verification and deduction.

This logic is enforced on the server side so that full premium data is never transmitted to the client unless the appropriate access state exists. The frontend uses the returned access flags to decide whether to display a full dataset or a paywall view. This mechanism protects the proprietary trade intelligence layer while still allowing users to preview value before purchasing access.

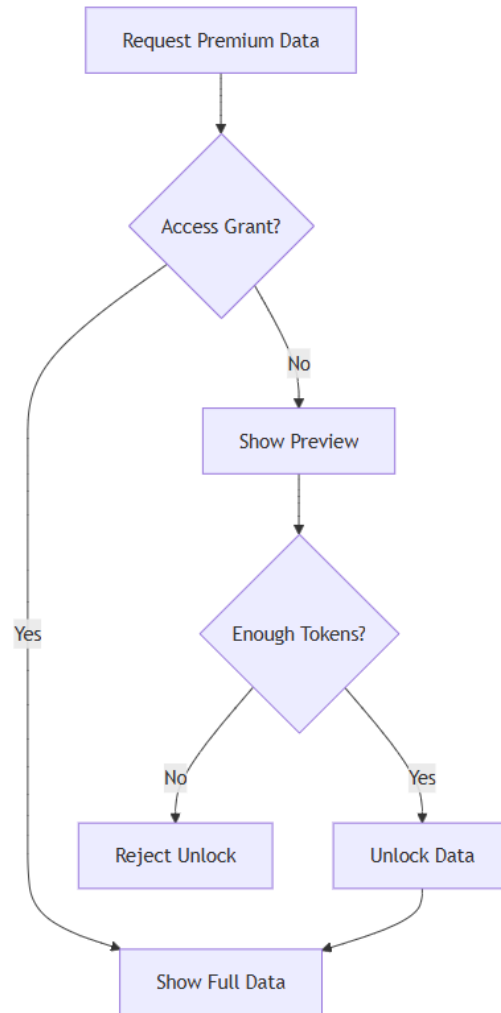


Figure 5.3: Token based access control workflow for premium data unlocking

5.1.5 Trade Directory and Contact Unlocking

The trade directory was developed to complement the transaction based intelligence engine with a curated collection of verified company profiles. Companies are classified by sector, role, and type, and may also include associated products and profile metadata. This allows users to browse a structured ecosystem of suppliers and buyers instead of relying only on raw transaction history.

A major feature of this module is the controlled exposure of contact data. Key decision maker records are stored separately from public company profile information and are masked by default. This allows the interface to indicate that premium contact data exists without exposing the sensitive details. When a logged in user chooses to unlock a contact, the system verifies token availability, records the unlock action, deducts the required tokens, and then reveals the protected details. This design connects the trade directory directly to the token economy while preserving a controlled and auditable access model.

5.1.6 Frontend and User Interaction Flow

The ZaraiLink frontend is implemented as a React based single page application that unifies search, market intelligence, directory browsing, watchlist management, and subscriptions within one interface. Shared providers such as `AuthContext`, `ThemeContext`, and the toast notification layer manage application wide state and user feedback. Reusable hooks such as the debounce hook and watchlist hook reduce duplicated logic and improve responsiveness during repeated actions.

The typical user journey begins at the dashboard, where a user enters an HS code, product name, or natural language query. Depending on the query type and context, the frontend routes the user to either the ranked search results page or the HS code dashboard. From there, the user can refine results, switch trade perspectives, inspect detailed company intelligence, compare entities, or move into the trade directory.

The platform also supports watchlist management, contact unlocking, and subscription based token top ups. This creates a continuous flow between broad market discovery and detailed company level intelligence. The frontend remains closely tied to backend access state so that the interface reflects the correct visibility level at each stage of interaction.

6. Testing and Results

6.1 Testing and Results

This section presents the testing process and observed results for ZaraiLink. The platform was evaluated through a combination of automated backend testing, script based validation, feature specific workflow checks, and direct inspection of system outputs. The purpose of this section is to show that the implemented system behaves correctly across its core modules, produces meaningful trade intelligence, and enforces access restrictions in a consistent manner.

6.1.1 Testing Approach

The testing strategy for ZaraiLink was designed to evaluate both the correctness of internal system logic and the behavior of end user workflows. A combination of automated backend testing, script based validation, and workflow oriented verification was used so that the platform could be assessed from multiple perspectives. This made it possible to test not only isolated components such as account logic, aggregation functions, and token deduction, but also complete flows such as search, dashboard generation, premium data restriction, and access unlocking.

Automated tests were used to validate important backend behaviors including user management, verification token handling, token balance operations, and selected API related logic. In parallel, targeted script based checks were developed and executed to evaluate trade data integrity, search aggregation outputs, Natural Language Understanding behavior, access control consistency, and entitlement state transitions. This combination ensured that both functional correctness and system level behavior could be examined in a controlled and repeatable manner.

The testing process focused on six major areas:

- backend unit and integration testing,

- data integrity validation,
- search aggregation and dashboard generation,
- natural language query interpretation,
- token based access control, and
- directory and contact handling workflows.

This approach allowed ZaraiLink to be evaluated at both the component level and the workflow level. It also produced measurable evidence such as pass counts, latency figures, record counts, aggregation outputs, and access state transitions, all of which provided a structured basis for assessing the technical reliability of the platform.

6.1.2 Automated Testing Results

The first layer of validation came from the existing automated backend test suite. The automated tests were executed to verify core application behavior related to authentication, user models, verification tokens, token balances, and transaction level business logic. These tests provided a direct measure of baseline system reliability.

The executed `pytest` suite produced a mixed but largely successful outcome. A total of 62 tests passed, while 4 tests failed. The successful cases validated important behaviors such as email based user creation, uniqueness constraints, verification token regeneration, token balance initialization, and token deduction operations. These results indicate that the essential account and monetization foundations of the platform are functioning correctly.

The failed cases were also informative. Two failures referenced a removed or outdated model related to user alert preferences, suggesting that the tests were not fully aligned with the latest codebase after refactoring. Two additional failures were associated with the subscription redeem endpoint, where the API returned a `400 Bad Request` response instead of the expected behavior. These failures do not undermine the core system functionality, but they highlight areas where test maintenance and endpoint validation can be improved.

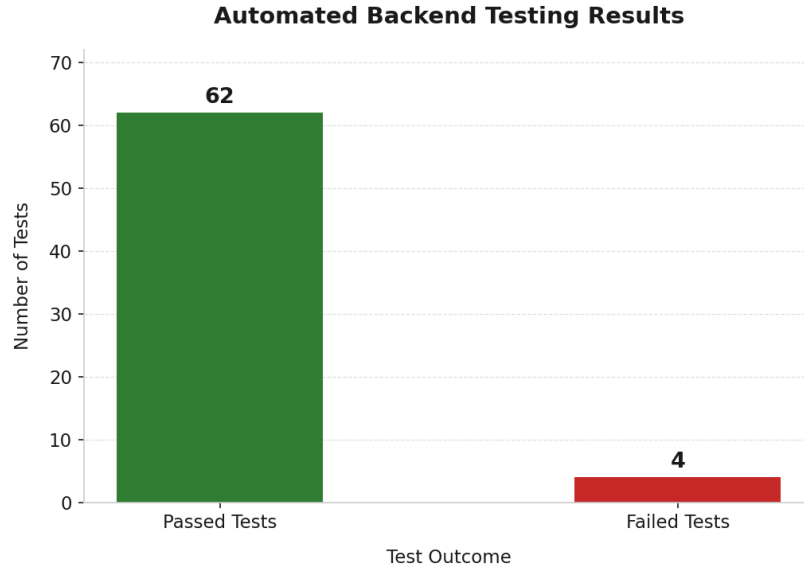


Figure 6.1: Summary view of automated backend testing results

Table 6.1: Summary of automated testing results

Test Area	Observed Result	Status
User model and authentication tests	User creation, uniqueness, verification token logic, and token initialization validated successfully	Passed
Token balance operations	Token deduction and balance update logic validated successfully	Passed
Subscription redeem endpoint tests	Endpoint returned unexpected 400 Bad Request responses in selected cases	Partial

6.1.3 Data Validation and Aggregation Results

The trade intelligence engine depends on the correctness of the underlying trade dataset. For this reason, data validation was treated as a separate result category. Script based checks were executed directly against the database to verify that the loaded transaction records matched expected categories and volumes.

The validation scripts confirmed that the transaction dataset had been loaded and mapped successfully. For example, the system identified 825 import transac-

tions from China within the relevant data slice used for testing. It also confirmed the existence of multiple dextrose related subcategories, including **Dextrose Anhydrous** under HS code 1702.3000, with 347 Chinese import records linked to that subcategory. These results show that the ETL and classification pipeline preserved both the trade records and the hierarchical commodity structure required by the application.

A second stage of validation focused on the aggregation engine. In this test, 2,232 import transactions were passed into the aggregation workflow. The engine then filtered the records by product subcategory and country, reducing the dataset to 348 relevant subcategory transactions and finally to 347 China specific records. These records were then grouped by trade entity and ranked into a supplier intelligence list. This demonstrated that ZaraiLink can transform large raw datasets into concise company level trade intelligence in real time.

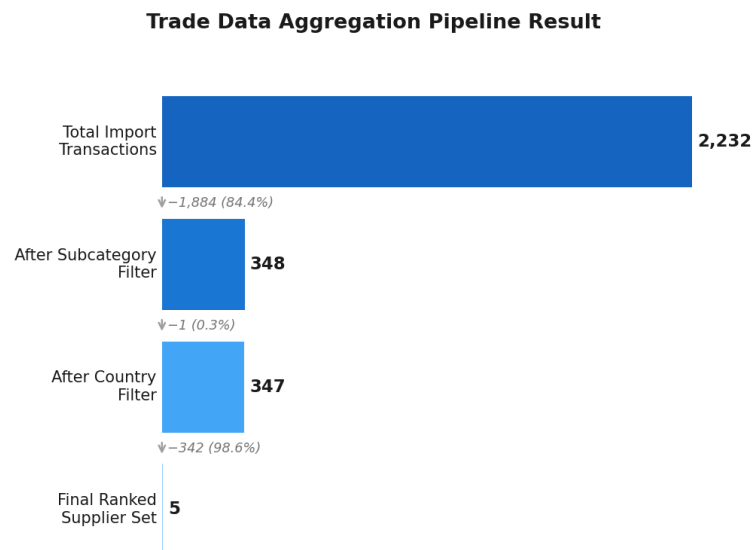


Figure 6.2: Aggregation result showing filtered and ranked supplier intelligence

Table 6.2: Observed data validation and aggregation results

Validation Check	Observed Result	Status
Chinese import transaction validation	825 relevant import transactions identified successfully	Passed
Dextrose subcategory mapping	Dextrose related product subcategories resolved correctly, including HS 1702.3000	Passed
Aggregation input volume	2,232 import transactions passed into aggregation workflow	Passed
Filtered aggregation output	348 subcategory rows and 347 China filtered rows produced correctly	Passed
Supplier ranking output	Top ranked supplier list generated successfully from grouped transactions	Passed

6.1.4 Natural Language Query Evaluation

The Natural Language Understanding pipeline is one of the central components of ZaraiLink because it allows users to search the platform using conversational trade queries instead of only rigid keyword inputs. The evaluation of this pipeline focused on both correctness and responsiveness under realistic, noisy, and ambiguous input conditions.

The broader project evidence indicated an overall Natural Language Understanding accuracy of 89.4% across 348 evaluated queries. To strengthen this result, an expanded scripted evaluation was performed on more than sixty additional queries. These queries were designed to cover:

- clean commercial phrasing,
- implicit trade intent,
- typographical variation,
- acronym based country references,
- malformed punctuation,

- mixed Roman Urdu and English input,
- long narrative requests, and
- contradictory query structure.

Intent Classification: Intent classification performed strongly across both direct and indirect phrasing. Queries such as `I want to buy sugar` and `We are exporting jaggery` were correctly mapped to buy and sell directions respectively. The system also handled implicit wording such as `Sourcing lactose from Europe` and `Distributing refined sugar abroad` without requiring explicit trade verbs.

The intent evaluation also showed the following behavior:

- ambiguous phrasing defaulted safely to a buy oriented interpretation,
- contradictory phrasing such as `I want to buy and sell molasses` still produced a valid directional result,
- the engine preferred returning a structured interpretation rather than failing outright.

Product Extraction: Product extraction demonstrated useful resilience. Clean product names such as `dextrose` and `molasses` were isolated successfully. Typographical variation was preserved rather than discarded, as seen in inputs such as `dextros` and `molases`. This is beneficial because the downstream search layer can then apply fuzzy matching against stored product names.

The product extraction stage also showed that:

- informal phrasing such as `want jaggery stuff` and `need some choc asap` did not break the pipeline,
- noisy product strings were still converted into usable search terms,
- extracted noun phrases were occasionally less precise in informal commercial language.

Country Normalization: Country normalization performed well on both acronyms and spelling variation. Inputs such as `UAE`, `brazil`, and `Indai` were normalized successfully to their intended country forms.

Observed strengths in this stage included:

- successful handling of country acronyms,

- resilience to spelling variation and typographical errors,
- stable behavior across short and moderate length queries.

Price and Quantity Constraint Extraction: Price and quantity extraction produced strong results. Queries containing expressions such as `under $500`, `max $500`, and `minimum 100 metric tons` were converted correctly into structured numeric filters.

This stage was particularly important because:

- clear numeric constraints were resolved through regex based handling,
- expensive large language model fallback was skipped when unnecessary,
- this reduced processing cost and improved response time for well formed trade constraints.

Edge-Case and Noisy Query Handling: The edge-case evaluation showed that the system remained stable under noisy and mixed language input. Roman Urdu and English phrasing such as `mujhe sugar chahiye from China` did not cause the engine to fail. The system still extracted a meaningful noun phrase and identified the relevant country. Punctuation heavy queries and long narrative style requests were also processed successfully.

The expanded evaluation highlighted the following:

- mixed language queries remained processable,
- punctuation heavy queries did not destabilize the pipeline,
- long narrative queries produced usable structured outputs,
- in one long narrative example, `Pakistani` was incorrectly prioritized over `Chinese` as the country reference.

This final case represents a meaningful limitation in multi-country narrative parsing, but it did not prevent the system from producing a structured result.

Latency Results: Latency measurements further supported the practical viability of the pipeline. The SetFit stage incurred a one time cold boot overhead of approximately 9,017 milliseconds when the model was first loaded into memory. After initialization, warm intent classification completed in roughly 27 to 48 milliseconds.

A full end to end evaluation of the query `I want to buy dextrose from China under $700` completed in approximately 89.2 milliseconds under warm conditions while correctly extracting the intent, product, country, and price filter. This indicates that the pipeline is suitable for repeated interactive use once the runtime environment has been initialized.

Taken together, these findings indicate that the ZaraiLink NLU pipeline is robust across standard trade queries, noisy phrasing, typographical variation, and mixed language input. The system also benefits from layered fallback logic and efficient regex based short-circuiting when full model inference is unnecessary.

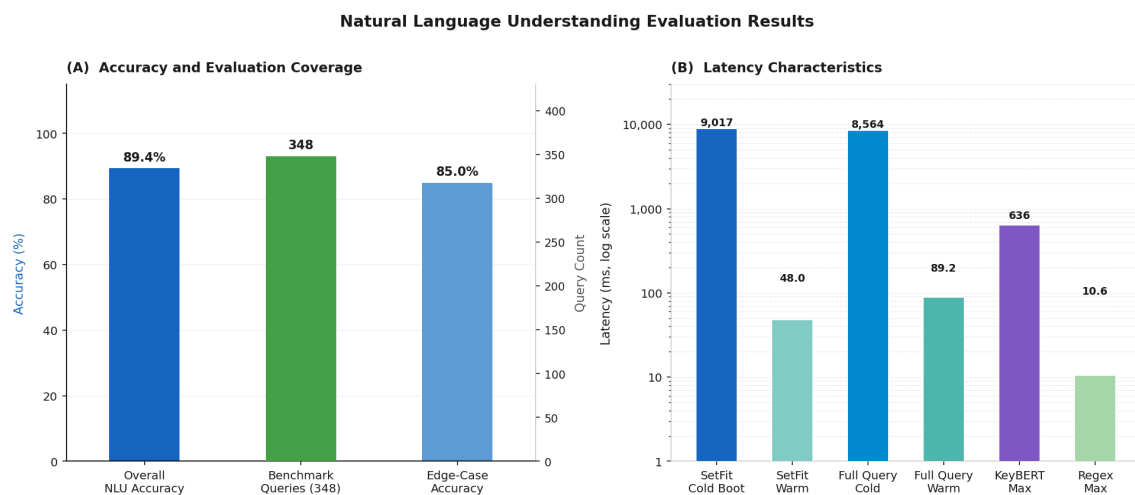


Figure 6.3: Observed accuracy and latency characteristics of the Natural Language Understanding pipeline

Table 6.3: Natural language query evaluation summary

Metric	Observed Value	Notes
Overall NLU accuracy	89.4%	Measured over 348 evaluated queries
Expanded edge-case evaluation	60+ queries	Included noisy, ambiguous, typo-rich, and mixed-language inputs
SetFit warm latency	27 to 48 ms	After initial model load
SetFit cold boot latency	9017 ms	One time startup overhead
Complex warm query latency	89.2 ms	Full end to end query interpretation
Cold full query latency	8564.1 ms	Includes initial model loading
Keyword extraction latency	275 to 636 ms	Observed for KeyBERT based extraction
Regex parsing latency	0 to 10.6 ms	Very fast numeric and text rule parsing
Country typo normalization	Successful	Correctly handled cases such as UAE, brazil, and Indai
Known narrative query limitation	Present	Multi-country long queries may prioritize the wrong country token

6.1.5 Access Control and Monetization Results

The token based paywall system is one of the most commercially important components of ZaraiLink. The testing of this subsystem focused on whether restricted users were prevented from seeing premium data, whether token insufficiency was handled correctly, whether successful unlock actions updated both balance and access state, and whether these rules remained consistent across all supported search paths. This part of the platform was tested across multiple products, HS code categories, and access states so that the reliability of premium data enforcement could be assessed more thoroughly.

A series of validation checks was performed against the access control logic using

different user balance states and different protected datasets. These checks confirmed that when a user did not possess access to a protected dataset, the backend consistently returned a `NO_ACCESS` state and restricted the response to preview level visibility. Unlock attempts made without sufficient tokens were rejected correctly, while valid unlock operations resulted in token deduction, access grant creation, and elevation of the user's access state to `FULL_ACCESS`. This behavior confirmed that the monetization workflow operates as an enforced backend policy rather than a superficial frontend restriction.

A major part of the evaluation focused on access consistency across different query routes. The aim was to verify that if a user did not possess access to a protected HS code or category, then the same restriction would apply regardless of how the dataset was reached. This was tested across:

- direct HS code searches,
- category and product name searches, and
- natural language trade queries.

For example, when access to HS code 1702.3000, representing glucose and glucose syrup, was not granted, the user was also correctly prevented from obtaining full access to related products such as `Dextrose Ball`. This restriction remained consistent whether the product was searched directly through its name, through the broader category, through the HS code path, or through a natural language request. The same consistency was observed in the opposite direction. Once access to the HS code or category had been purchased, the related products became accessible across all supported search modes.

This result is important because it confirms that ZaraiLink applies access control at the data and entitlement level rather than only at the interface level. Users are not able to bypass restrictions by switching from one search mode to another. The consistency of this behavior was tested across multiple products and categories and was observed to function correctly in all evaluated cases.

The overall monetization validation therefore demonstrated four important outcomes:

- restricted users were limited to preview level visibility,
- unlock attempts without sufficient tokens were denied,
- successful purchases updated both token balance and entitlement state, and

- access permissions remained consistent across HS code, product, category, and natural language search paths.

These results confirm that the paywall is not only a frontend visual layer but a backend enforced access control mechanism. Full premium data is not transmitted to the client unless the required access grant already exists. This behavior is fundamental to the platform’s monetization model and is especially important for future integration with production payment systems such as Stripe, where strict consistency between purchase state and data visibility will be essential.

Table 6.4: Access control and monetization validation

Scenario	Observed Result	Status
Restricted user requests premium data	Backend returned <code>NO_ACCESS</code> and restricted preview behavior	Passed
Unlock attempt with insufficient tokens	Request rejected due to insufficient balance	Passed
Unlock attempt with sufficient tokens	Tokens deducted and access grant created successfully	Passed
Post unlock access state	Access elevated to <code>FULL_ACCESS</code> for subsequent requests	Passed
Cross-query access consistency	Access restrictions remained consistent across HS code, product-name, category, and natural language search paths	Passed

6.1.6 Trade Directory and Workflow Validation

The trade directory and related company workflows were validated through a mixture of backend inspection, feature level verification, and frontend code review. The `CompanyViewSet` correctly supports role based filtering so that supplier and buyer directories can be separated at the API level. The frontend interfaces for `FindSuppliers` and `FindBuyers` were also confirmed to map user selected filters to the expected API parameters.

Contact visibility behavior was also validated. The `KeyContact` model supports conditional masking through the `is_public` flag and unlock state logic. Where a con-

tact is not public and no unlock record exists, the API is designed to return masked information instead of raw contact details. This verifies that the trade directory does not expose premium contact information without explicit authorization.

Watchlist behavior was assessed at the frontend infrastructure level. The feature is implemented through a custom React hook that persists company identifiers in browser local storage. This means that the functionality works for the local user session, but it is not yet synchronized across devices because there is no dedicated backend watchlist model. This should be treated as a valid implemented feature with a known architectural limitation.

6.1.7 Observed Limitations During Testing

The testing process confirmed that the major platform workflows of ZaraiLink are functioning correctly, while also revealing a small number of limitations that should be acknowledged for completeness. These limitations were localized and did not affect the overall stability of the system or the correctness of its core search, aggregation, and access control behavior.

A small number of backend tests remained unresolved at the time of evaluation. Four automated tests failed, but these failures were limited in scope. Two were associated with outdated test references following recent refactoring, while two were related to redeem endpoint behavior returning an unexpected **400 Bad Request** response. These issues indicate that parts of the automated test suite require alignment with the latest version of the codebase rather than suggesting failure of the platform as a whole.

The Natural Language Understanding pipeline also showed a few refinement areas under edge-case conditions. In particular, some fuzzy country matching cases and keyword extraction outcomes were less reliable in unusually noisy or narrative phrasing. For example, **america** could be mapped incorrectly in isolation, and long multi-country narrative queries could prioritize the wrong country token. These cases were relatively infrequent and did not prevent the system from producing structured outputs, but they highlight opportunities to improve country prioritization and semantic disambiguation in future versions.

A further limitation concerns watchlist persistence. The watchlist feature currently functions through frontend local storage, which supports normal single-user interaction within the same browser environment. This implementation is sufficient for the current project scope, but it does not yet provide backend synchronized persistence across devices or sessions. Extending this feature into a database-backed model would improve continuity in a production deployment.

These limitations are important to document, but they should be understood in context. The observed issues were limited to edge cases, test maintenance, or future scalability concerns. They did not undermine the successful operation of the platform’s primary capabilities, which were validated extensively through automated testing, script based evaluation, and workflow level verification.

6.1.8 Summary of Results

The testing results demonstrate that ZaraiLink successfully meets its primary platform objectives. The backend test suite validated the reliability of major account and token operations, while targeted validation scripts confirmed the correctness of the trade dataset, the aggregation engine, the Natural Language Understanding workflow, and the token based paywall. The system was able to transform large trade datasets into ranked supplier intelligence, interpret conversational search queries with strong observed accuracy, and enforce premium access restrictions consistently at the backend level.

The evaluation also showed that ZaraiLink performs well across multiple query styles, access states, and product discovery paths. Search interpretation remained robust under standard, noisy, and mixed language input. Aggregation logic reduced thousands of raw transaction records into meaningful company-level outputs. Access control behavior remained consistent across HS code, category, product, and natural language search routes. These outcomes indicate that the platform is not only feature complete at the project level but also operationally coherent across its core modules.

A small number of limitations were identified, but they were narrow in scope and did not affect the overall credibility of the system. Most importantly, the results show that ZaraiLink functions as a technically sound and commercially meaningful trade intelligence platform, with a strong foundation for future extension, deployment, and payment integration.

7. Conclusion and Future Work

In this project, we successfully designed and developed ZaraiLink, a trade intelligence platform aimed at improving access to structured import and export insights for businesses, traders, and market participants in Pakistan. The system combines a hierarchical trade data model, a flexible search engine, natural language query interpretation, a verified trade directory, and a token based access control mechanism into one integrated platform. Through this work, we were able to transform large volumes of raw trade records into usable business intelligence that supports supplier discovery, buyer discovery, market comparison, and product level analysis.

A major contribution of the project is the development of a dual mode search experience that supports both structured and conversational querying. Users can search through HS codes, product names, or natural language requests and receive relevant supplier or buyer intelligence through ranked results and market dashboards. The system also introduces a monetization layer through subscription plans, token balances, controlled data access, and contact unlocking. This makes the platform not only technically functional but also commercially aligned with how premium trade intelligence services operate in practice.

The project also demonstrated the importance of combining deterministic retrieval with machine learning based interpretation. By integrating modules such as SetFit, KeyBERT, RapidFuzz, and fallback parsing logic, ZaraiLink can handle both precise and noisy user queries while remaining efficient and robust. The curated trade directory further extends the usefulness of the system by allowing users to move beyond transaction records into verified business profiles and key contact discovery. Together, these features establish a strong foundation for a practical business intelligence system tailored to the needs of Pakistan’s trade ecosystem.

There are several important directions for future work. A natural extension of the platform is the development of a mobile application for field level traders and small and medium enterprises so that trade intelligence can be accessed more conveniently in operational settings. Urdu language support would also greatly improve accessibility for local users and make the platform more inclusive for a broader business

audience. In addition, future versions of ZaraiLink can incorporate real time price alerts and market movement notifications so that users are informed immediately when important shifts occur within their product categories of interest.

Another promising direction is the addition of predictive analytics for demand forecasting and price trend analysis. By combining historical trade data with time series modelling and market behavior signals, the platform could move from descriptive intelligence toward forward looking decision support. The monetization layer can also be strengthened through direct integration with production payment infrastructure such as Stripe, allowing real world subscription management and secure token purchases. Further improvements may include richer semantic search, expanded trade datasets, stronger multilingual query handling, and backend synchronized watchlist persistence across devices.

In conclusion, ZaraiLink shows the potential of combining trade data engineering, natural language search, and controlled premium access into a single platform tailored for commercial use. It bridges the gap between raw customs style records and actionable trade intelligence, while also laying the groundwork for future expansion into mobile accessibility, predictive analytics, and real time market support. The project demonstrates that an AI assisted trade intelligence platform can be both technically viable and practically valuable for local and international trade stakeholders.

8. Reflection

8.1 Reflection

This section includes reflection on our Final Year Project journey, signifying the learning of the team on a collective and individual level, highlighting our growth in the respective domains we worked in and how we were able to bring out the best from each other.

8.1.1 Individual Reflection

Umar Kashif

Working on the backend and database layer of ZaraiLink was both challenging and deeply rewarding. My primary responsibility was designing and maintaining the Django REST Framework application, structuring the PostgreSQL schema, and ensuring that all data models correctly reflected the business logic of a cross-border trade platform. Early on, getting the database design right was harder than I anticipated. Trade data has a lot of hierarchical relationships, and making sure the schema could support the token system, user access grants, and company profiles without becoming a mess took several iterations. Authentication and session management with CSRF and CORS for a React SPA added another layer of complexity I had not dealt with at this scale before.

The turning point for me was when Salman and I sat down together to align the backend search API with the NLU pipeline output. Bridging the gap between what the ML service was producing and what the frontend needed to display required us to go back and forth on the data contracts quite a bit, but once everything clicked into place and the search functionality was working end-to-end, returning real, ranked trade results from natural language queries, it felt like the entire project came alive. That moment made all the earlier frustration worth it. This project taught me how

to think about backend systems not just as code but as a foundation that every other part of the product depends on.

Salman Adnan

My work on the NLU and ML pipeline was the most intellectually demanding experience of my academic journey so far. I was responsible for building the entire natural language understanding engine that sits at the core of ZaraiLink's search, from intent classification using SetFit, to entity extraction with GLiNER and KeyBERT, to country normalization with RapidFuzz, and finally the OpenRouter fallback for complex queries the local models could not resolve confidently. Coming into this, I had a theoretical understanding of NLP but had never trained and deployed a pipeline of this complexity in a real product.

The early stages were rough. Getting SetFit to train meaningfully required generating and cleaning hundreds of labeled trade queries, and the first few runs produced accuracy numbers that were not good enough. Debugging why the pipeline was failing on certain query types, particularly ones with ambiguous price signals, took a lot of iteration. But the breakthrough came gradually. As each component of the pipeline matured and connected properly, the accuracy climbed and latency dropped to acceptable levels. The proudest moment for the entire team, and for me personally, was seeing the NLU search handle all sorts of real-world trade queries correctly, informal phrasing, mixed languages, abbreviations and all. Collaborating with Umar to get that pipeline fully integrated with the backend was where the project really came together.

Fahad Nadeem

My responsibility was the React frontend, and this project pushed my frontend development skills further than any coursework ever had. I was in charge of designing and building every user-facing module, the Dashboard, the Search interface, the Trade Directory, the Subscriptions flow, and the Accounts section. Beyond just making things functional, there was a strong emphasis on the product feeling professional and polished, which meant spending serious time on layout, responsiveness, and interaction design using TailwindCSS and Framer Motion.

The most significant moment for me was the UI redesign for the platform ahead of our Kaavish 2 presentation. I reworked the interface to a level the whole team was genuinely proud of. It looked and felt like a real product, not a university project. Getting the UI exactly where we wanted it, and seeing the team's reaction, was a highlight of the entire experience. The challenge throughout was keeping the

frontend in sync with a backend and NLU pipeline that were constantly evolving. API response structures would change, new fields would appear, and I had to adapt quickly. It taught me the importance of clear communication between frontend and backend, and by the end I had a much stronger instinct for how to build UIs that are robust to change.

Ibrahim Rana

My domain was DevOps and deployment infrastructure, which meant owning everything from Docker containerization to environment configuration and making sure the full stack was consistently runnable across different machines. On paper, this sounded straightforward. In practice, it was anything but. Getting the Django backend, Redis, and the React frontend all playing nicely together inside Docker, writing setup scripts that actually worked across both Windows and Linux environments, and debugging the inevitable port conflicts and environment variable issues took far more time than I had anticipated. There were moments where the infrastructure felt like it was actively fighting back.

But persistence paid off. The moment the entire system spun up cleanly from a single command, backend, frontend, and Redis all running together without manual intervention, was genuinely satisfying. It is the kind of thing that sounds boring to explain but matters enormously in practice, because without stable infrastructure none of the other work could be properly tested or demonstrated. This project gave me a real appreciation for DevOps as a discipline in its own right, and I came away with practical skills in containerization and deployment that I know will be directly useful in industry.

8.1.2 Team Reflection

Our collaboration as a team was chaotic at first, stressful throughout, and ultimately one of the most fulfilling experiences of our time at university. Each of us came in with a defined primary domain, Umar on backend and database, Salman on the NLU pipeline, Fahad on frontend, and Ibrahim on DevOps and deployment infrastructure, but the nature of building an integrated product meant we were constantly in each other's territory, helping debug, review, and align.

Kaavish 1 was a wake-up call. We underestimated how much coordination a project of this scope required, and the pressure of that first presentation exposed gaps in our communication and planning. It was stressful and at times disorganised. But that experience forced us to recalibrate. By Kaavish 2, we had found our rhythm.

We became more deliberate about syncing up, clearer about who owned what, and more willing to ask for help early rather than sitting on blockers.

All of our communication happened over WhatsApp, which kept things informal and fast. It meant conversations happened in real time, decisions were made quickly, and no one was ever out of the loop for long. The defining moment of our teamwork was when the NLU search went fully end-to-end, Umar’s backend, Salman’s pipeline, Fahad’s interface, and Ibrahim’s infrastructure all working together at once. That moment belonged to all of us equally. We came into this project as four individuals with different skill sets and left as a team that knew how to build something real together.

8.1.3 Process Reflection

Understanding each other’s working styles, schedules, and strengths proved to be one of the more human challenges of this project. Umar, as team lead, carried the heaviest workload and was often the last one debugging at night. Salman, serving as second in command, worked best in the early hours and brought structure and discipline to the team’s technical direction. Fahad, while managing a significant course load, consistently pushed through as well, ensuring he met expectations and continuously iterated on the UI to achieve the best possible result. These differences in availability and working rhythm occasionally created coordination friction, but we adapted around each other over time.

With Umar leading and Salman supporting, decision-making was efficient when it needed to be, but the structure also meant that ideation was sometimes concentrated in one or two people rather than emerging from the whole team. This is perhaps our most honest area of self-criticism: too often, the direction of the project was driven by a subset of the team while others executed. In hindsight, we would have benefited from more deliberate efforts to involve everyone in the thinking, not just the doing.

Several areas stood out as opportunities for growth. First, we lacked a clear early picture of each other’s strengths and weaknesses, which meant task allocation in the beginning was more assumption-based than it should have been. Second, while WhatsApp kept us connected, our communication around system design decisions was often informal and underdocumented. Things were decided in conversation and not always shared with the full team. Third, our collective understanding of the overall system architecture took time to develop. Early on, each member understood their own component well but had only a surface-level grasp of how the pieces fit together. More structured knowledge-sharing sessions, where each member walked the team through their module, would have built stronger collective ownership and

made cross-functional debugging far less painful.

8.1.4 Plans vs Achievements

What was Proposed vs. What was Achieved

ZaraiLink went through more pivots than most projects, and being honest about that is important. The original vision was ambitious: a trade intelligence platform powered by Graph Neural Networks (GNNs), capable of predictive pricing, predictive trade volume forecasting, buyer and seller classification, and a full trade directory, all driven by graph-based relational modelling of cross-border trade data. It was, in retrospect, a scope that reflected our enthusiasm more than our constraints.

After multiple meetings with our industry partner PAR, the requirements became clearer and more grounded. The platform was restructured around three core modules: Trade Lens (a product analytics dashboard), Trade Ledger (a company-level dashboard), and a Trade Directory for supplier and buyer discovery. At this stage, natural language search was not part of the plan at all.

The most significant pivot came at the end of Kaavish 1, when conversations with our supervisors made it clear that applying GNNs to this problem was not the right call. The data relationships in our trade dataset did not have the graph structure that would justify GNN complexity, and forcing that architecture would have added technical overhead without meaningful academic or practical benefit. We made the decision to drop GNNs entirely and redirect that energy toward building a rigorous NLU-powered search engine, a pivot that proved to be the right one. The search pipeline became the technical core of ZaraiLink and the feature that most impressed both our supervisors and our industry partner.

By Kaavish 2, we had successfully delivered:

- A fully functional NLU search pipeline supporting natural language trade queries with intent classification, entity extraction, and LLM fallback.
- A token-based subscription and access system.
- A Trade Directory with supplier and buyer discovery.
- A Deal Details page consolidating all trade intelligence into a clean four-tab interface.
- HS Code search and category-based search, added in response to PAR's user feedback.

- A professional, responsive React frontend redesigned to meet PAR’s usability standards.

Features Removed or Not Implemented

Two significant structural changes were made based on feedback from PAR during the final phase of the project:

1. **Trade Lens and Trade Ledger Modules:** These two dashboards were removed after PAR flagged that the platform felt too complex for their target users, traders who needed quick, actionable information rather than deep analytical dashboards. In response, we consolidated the most valuable information from both modules into a single Deal Details page with four focused tabs. This was the right decision. It made the product cleaner, faster to navigate, and far more aligned with how actual traders interact with trade data.
2. **GNN-based Predictive Models:** The originally proposed Graph Neural Network architecture for price and volume prediction was dropped after supervisor consultation revealed that the graph structure assumption did not hold for our dataset. Applying GNNs would have introduced unjustified complexity without improving outcomes. The decision to pivot toward a hybrid NLP pipeline instead produced a more defensible, more rigorous, and more practically useful system.

Design Decisions: Opportunities and Challenges

- **Opportunity:** The pivot to NLU search at the end of Kaavish 1, while stressful at the time, opened up the most technically interesting part of the project. Building a multi-stage pipeline combining SetFit, GLiNER, KeyBERT, RapidFuzz, and selective LLM calls gave the team genuine exposure to production ML engineering, something no coursework had prepared us for.
- **Opportunity:** The ongoing engagement with PAR throughout the project, while sometimes requiring us to redo work, ultimately made ZaraiLink a better product. Their insistence on simplicity and usability pushed us to make design decisions we might not have made on our own.
- **Challenge:** The late addition of HS Code and category-based search required extending the search architecture that had been built around natural language input. Integrating a structured lookup pathway alongside the NLU pipeline

without breaking existing functionality required careful backend work and close coordination between Umar and Salman.

- **Challenge:** Scope changes arriving late in the development cycle, particularly the removal of Trade Lens and Trade Ledger, meant that Fahad had to redesign significant portions of the frontend under time pressure. Managing that without compromising the quality of the final UI required the kind of focused effort that only comes from genuine investment in the product.

References

- [1] Yinqiong Cai et al. “Semantic Models for the First-stage Retrieval: A Comprehensive Review”. In: *Institute of Computing Technology, Chinese Academy of Sciences; Alibaba Group* (2023).
- [2] Maarten Grootendorst. *KeyBERT: Minimal Keyword Extraction with BERT*. GitHub repository. 2020.
- [3] Rada Mihalcea and Paul Tarau. “TextRank: Bringing Order into Texts”. In: *Association for Computational Linguistics* (2004).
- [4] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *arXiv preprint arXiv:1908.10084* (2019).
- [5] Lewis Tunstall et al. “SetFit: Efficient Few-Shot Learning Without Prompts”. In: *arXiv preprint arXiv:2209.11055*. 2022.
- [6] Urchade Zaratiana et al. “GLiNER: Generalist Model for Named Entity Recognition using Bidirectional Transformer”. In: *arXiv preprint* (2023).